
EiKI Summary

Moritz Gerhardt



Table of Contents

1	What is AI?	3	4	Local and Adversarial Search	20
1.1	What is Intelligence?	3	4.1	Optimization Problems	20
1.1.1	Turing Test	3	4.1.1	Terminology	20
1.1.2	Chinese Room Argument	3	4.2	Local Search	21
1.1.3	Does it even matter?	3	4.2.1	Hill Climbing	21
1.2	Characteristics of AI	4	4.2.2	Gradient Descent	22
1.3	Foundations of AI	4	4.2.3	Beam Search	24
			4.2.4	Simulated Annealing	24
2	AI Systems	5	4.3	Adversarial Search	25
2.1	What is an AI System?	5	4.3.1	Games	25
2.1.1	Environment	5	4.3.2	Games vs. Search Problems	26
2.1.2	Agents	6	4.3.3	Minimax Algorithm	26
2.1.3	Types of Agents	7	4.3.4	Alpha-Beta Pruning	28
2.2	Creating Intelligent Agents	9			
2.2.1	Search Algorithms	9	5	Constraint Satisfaction Problems	31
2.2.2	Reinforcement Learning	9	5.1	Assignment of Values to Variables	31
2.2.3	Genetic Algorithms	10	5.2	Constraint Graphs	32
			5.3	Types of Constraints	32
3	Uninformed and Informed Search	11	5.4	Solving CSPs: Search	33
3.1	Problem Solving	11	5.4.1	Naive Search	33
3.1.1	Key Terminology	11	5.4.2	Backtracking Search	33
3.1.2	Tree Search Algorithms	12	5.5	Heuristics for CSPs	34
3.2	Uninformed Search	13	5.6	Constraint Propagation	34
3.2.1	Uniform-Cost Search (UCS)	13	5.6.1	Forward Checking	35
3.2.2	Breadth-First Search	14	5.6.2	Maintaining Arc Consistency (MAC)	35
3.2.3	Depth-First Search (DFS)	14	5.6.3	Path Consistency	36
3.2.4	Depth-Limited Search (DLS)	15	5.6.4	k-Consistency	36
3.2.5	Iterative Deepening Search (IDS)	15	5.6.5	Constrain Propagation & Backtracking Search	36
3.3	Heuristics h	16	5.7	Local Search for CSPs	37
3.3.1	Relaxed Problems	16	5.8	Problem Decomposition	37
3.3.2	Dominance	16	5.9	Tree-Structured CSPs	37
3.3.3	Combining Heuristics	17	5.9.1	Linear Algorithm	37
3.4	Informed Search	17	5.9.2	Nearly Tree-Structured Problems	37
3.4.1	Greedy Best-First Search	17			
3.4.2	A* Search	18	6	Logic & AI: Propositional Logic	39
3.4.3	Alternatives to A*	18	6.1	Logic	39
3.4.4	Graph Search	19			

6.2	Syntax	40	11 Machine Learning & Neural Networks	60
6.3	Semantics	40	11.1 Learning	60
6.4	Tautology	40	11.1.1 Methods of Learning	60
6.5	Logical Equivalence	40	11.1.2 Inductive Learning	60
6.6	Inference / Entailment	41	11.2 Machine Learning	60
6.6.1	Principle of Non-Contradiction . .	41	11.2.1 Machine Learning & Human	
6.7	Conjunctive Normal Form (CNF)	41	Learning	60
6.8	Modus Ponens	42	11.2.2 Designing a Learning System . . .	61
6.8.1	Unit Resolution	42	11.2.3 Types of Learning	61
6.8.2	General Resolution	42	11.2.4 Supervised Learning	62
6.9	Resolution	42	11.2.5 Representation	62
6.9.1	Resolution Algorithm	42	11.2.6 Feature Engineering	62
6.10	Limitations of Propositional Logic	43	11.3 Evaluating a Model	63
			11.3.1 Overfitting	63
7 Logic & AI: First-Order Logic	44		11.4 Neural Networks	64
7.1 Elements in FOL	44		11.4.1 Deep Learning	64
7.2 Quantifiers	44		11.4.2 Perceptron	64
7.3 Substitution	45		11.4.3 Perceptron to Neural Network . .	66
7.3.1 Instantiating Quantifiers with			11.5 Forward Propagation	66
SUBST	45		11.5.1 Applying a NN	66
7.4 Generalized Modus Ponens	46		11.6 Backpropagation	67
7.4.1 Unification	46			
7.5 First-Order Conjunctive Normal Form . .	46	12 Reinforcement Learning	70	
7.6 Prolog	46	12.1 Markov Decision Process (MDP)	70	
7.6.1 Atoms and Variables	47	12.1.1 Solve MDPs	71	
7.6.2 Prolog as a Database	47	12.1.2 Optimal Utility	71	
7.7 Gödel's Incompleteness Theorem	47	12.2 Reinforcement Learning	72	
		12.2.1 Exploration	73	
8 Uncertainty	48	12.2.2 Deep Q-Networks (DQN)	74	
8.1 Probabilities	48	12.2.3 Policy Search	74	
8.1.1 Basics	48	12.3 Deep Neural Networks (AlphaZero)	74	
8.1.2 Kolmogorov's Axioms of Probability	49	12.3.1 Training Objective: Loss For-		
8.1.3 Random Variables	49	mulization	75	
8.1.4 Propositions	50			
8.2 Joint Distribution	50			
8.2.1 Marginalization	50			
8.2.2 Conditional Probabilities	51			
8.2.3 Independence	51			
8.2.4 Bayes Theorem	51			
8.3 Uncertainty in AI	52			
9 Bayesian Networks	53			
9.1 Naïve Bayes	54			
9.2 Inference in Bayesian Networks	54			
9.2.1 Variable Elimination	54			
9.2.2 Approximate Inference by Stochastic				
Sampling	55			
10 Machine Ethics	59			
10.1 Concerns	59			
10.1.1 Biases	59			
10.1.2 Data Privacy	59			
10.1.3 Causal Reasoning without Causality	59			
10.2 Mitigation of Concerns	59			
10.2.1 Make AI systems aware of un-				
wanted behaviour	59			

1 What is AI?

In general, there is no set definition of what artificial intelligence is. The most common definition is:

A system of machines that can perform tasks that normally require human intelligence.

This is flawed in that intelligence itself also does not have a set definition, and neither does human intelligence.

1.1 What is Intelligence?

1.1.1 Turing Test

Intelligence is often defined by the Turing Test. Hereby it is assumed that an entity is "intelligent" if it cannot be distinguished from another intelligent entity by observing its behavior.

The Turing Test is set up like this:

1. Human interrogator interacts with two entities, A and B. Hereby one of the two is already assumed to be intelligent (another human).
2. If the interrogator cannot distinguish which entity is the assumed intelligent entity, it is assumed that the other entity is intelligent as well.

This test is flawed as it does not distinguish between different intelligence levels (knowledge, reasoning, language understanding, learning etc.). It merely assumes "intelligence" based on how human-like its conversational performance is. It also is very subjective to the interrogator. It is not based on an objective metric and therefore the outcome can differ from person to person.

It operationalizes intelligence based on proximity to human linguistic behavior, which can be seen as "human-centered" and thus incomplete. Just because humans are intelligent, does not mean that all intelligent entities must be human-like.

1.1.2 Chinese Room Argument

The Chinese Room Argument tries to answer the question whether intelligence is the same as intelligent behavior. It assumes that even if a machine behaves intelligently, that does not mean it is actually intelligent.

The argument goes as follows:

1. A person who doesn't know chinese is put into a room. Outside the room there is a person, who can only interact with this person by slipping them notes written in chinese.
2. The person inside the room has detailed instructions as to how to answer the notes, without any translation or understanding.
3. To the person outside of the room it looks like the person inside is able to understand and answer to these notes, therefore the person outside of the room assumes that the person inside knows chinese.

So the person outside of the room assumes intelligence due to behavior, that does not necessitates the intelligence if the answers are already known.

Is a self-driving car intelligent? Are Large-Language Models intelligent?

1.1.3 Does it even matter?

The question of what intelligence is a long, complex and difficult question. However, as it is a more philosophical question it doesn't actually really affect the scientific field of AI. It's a definition, not a basis.

1.2 Characteristics of AI

AI's are often divided into two categories: general AI and narrow AI.

General & Narrow AI

- General (strong) AI is defined so that it can handle any intellectual task
- Narrow (weak) AI is defined so that it has a specific task or domain it works in.

This mirrors the distinction between intelligence and acting intelligent. Most (if not all) deployed systems are considered narrow (e.g. LLMs: excel in natural language processing, performance drops in other domains). General AI is the long term research goal of some AI researchers.

An AI should possess the following characteristics:

Adaptability

The ability to improve performance by learning from experience

Autonomous

The ability to perform tasks without constant guidance from a user / expert

Usually an AI is modelled after one of the two following concepts:

Law of Thoughts

Systems that think rationally: Using formal logic to derive correct conclusions from a set of premises. Based on the idea that human reasoning can be formalized and mechanized.

Rational Behavior

Systems that act in a way that maximizes the expected utility or value given the available information. Based on a mathematical model of rationality that achieves the most desirable outcome given the information available.

Rationality has two advantages

- **More General:** For many situations a provable correct option does not exist. The most likely outcome is a better solution.
- **More amenable:** Rationality can be defined, refined and optimized.

Rational-agent systems simplify real environments, thus creating possible inadequacies. However, they offer a clear normative criterion by selecting the actions expected to maximize the performance measure.

Systems that think and behave like humans are often talked about in the scientific field of **Cognitive Science**.

1.3 Foundations of AI

The field of AI is made up by many other fields:

- **Philosophy:** Logic, reasoning, mind as a physical system, foundations of learning, language, rationality
- **Mathematics:** Formal representation and proof algorithms, computation, decidability, tracability, probability
- **Psychology:** Adaptation, phenomena of perception and motor control
- **Economics:** Formal theory of rational decisions, game theory
- **Linguistics:** Knowledge representation, grammar
- **Neuroscience:** Physical substrate for mental processes
- **Control theory:** homeostatic systems, stability, optimal agent design

2 AI Systems

2.1 What is an AI System?

An AI System can be defined as the study of **rational agents** and **their environments**



2.1.1 Environment

- "The surroundings or conditions in which a person, animal, or plant lives or operates" - Oxford Dictionary
- In AI: The surrounding of an (AI) agent, where the agent operates
- Does not have to be real - Can be artificial
- Example:
 - Selfdriving cars: Street, traffic, weather, road signs, ...
 - Chess: Board, pieces,...

Characteristics of Environments

Discrete vs. Continous

Discrete: Environment has countable number of distinct, well defined states

Continous: Not discrete - Uncountable number of states

For Example:

- Discrete: Chess - Every state of the board is mathematically determinable and defined
- Continous: Selfdriving car - Practically infinite positions and conditions

Observable vs. Partially Observable / Unobservable

Observable: State is completely determinable at each point in time

Partially Observable: State is only partially determinable or only determinable at specific points in time

Unobservable: State is never determinable

Static vs. Dynamic

Static: Environment does **not** change while the agent is acting

Dynamic: Environment can change while the agent is acting

For Example:

- Static: Jigsaw puzzle - State does not change without the agents action

- Dynamic: Driving - State still changes even when the agent stops acting

Single Agent vs. Multiple Agents

Single Agent: Environment contains only one agents

Multiple Agents: Environment can contain multiple agents

Deterministic vs. Stochastic / Probabilistic

Deterministic: Next state is completely determined by the current state and the actions of the agents

Probabilistic: Next state is also influenced by other factors (e.g. random events)

Episodic vs. Non-episodic / Sequential

Episodic: Each **episode** consists of the agent perceiving and then acting. Every action is dependent only on the **current** episode

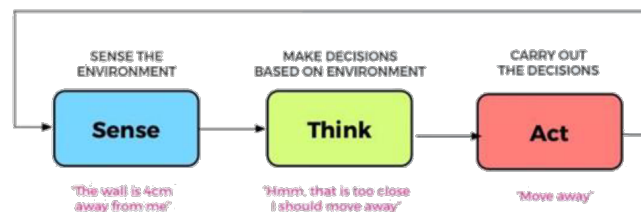
Non-episodic / Sequential: Actions are also dependent on past states and actions.

These characteristics are important, as the environment specifies the specific needs of the agent:

- Different environments require different agent designs
- Not every algorithm works for a specific environment

2.1.2 Agents

- **Sense:** Perceives its environment
- **Think:** Makes decisions autonomously
- **Act:** Acts upon the environments



Rules of AI Agents

1. Must be able to perceive its environment
2. Observations must be used to make decisions
3. Decisions should be used to act
4. (Action should be rational)

Rational Agents

Selects the action that is expected to maximize its performance measure, given its percept sequence, prior knowledge, and available actions.

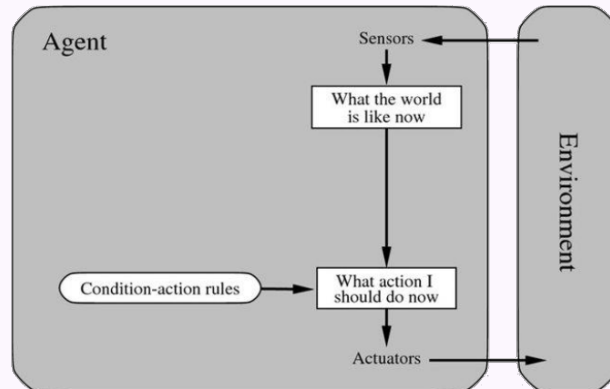
Performance is hereby often measured by a function that evaluates a sequence of actions. This function is task-dependent and cannot be generalized.

2.1.3 Types of Agents

Reflex Agent

Act only on the basis of the current percept, ignores past memory

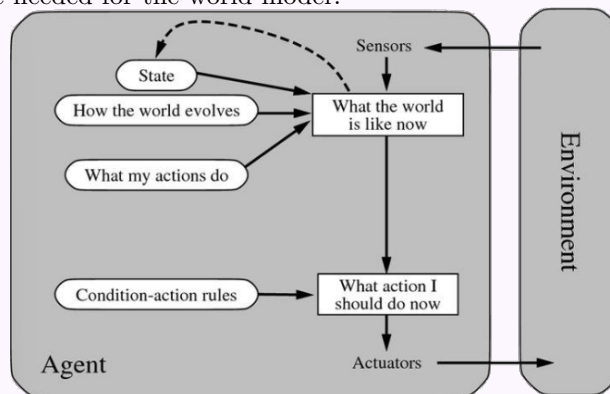
- Implemented through condition-action rules - map state to action
- Problem:
 - Limited decision making
 - No knowledge about anything that cannot be currently perceived
 - Hard to handle in complex environments



Model-Based Agent

Similar decision making to reflex agent, but keep track of the world state

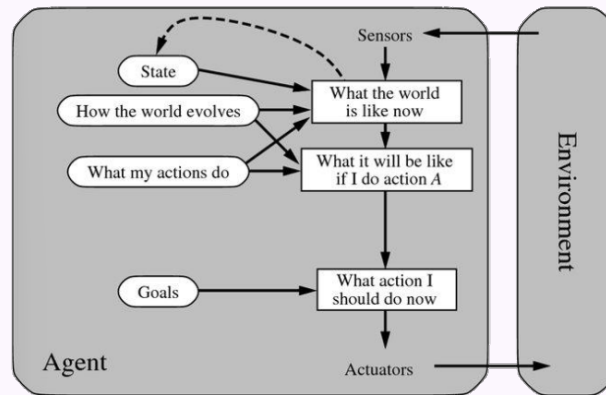
- Input is interpreted and mapped to an internal state representation of the world
- Problem:
 - How do these action affect the internal representation of the world?
 - What details are needed for the world model?



Goal-Based Agent

Essentially a Model-Based Agent with additional functionality that stores desirable states

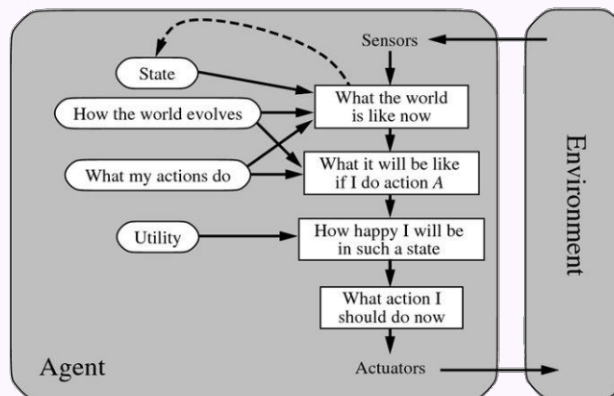
- Knows what states are desirable and acts towards them
- Problem:
 - Difficult to choose actions if a lot of actions are required to achieve a goal



Utility-Based Agent

Similar to Goal-Based Agents, but instead of providing goals it provides a utility function for rating actions and scenarios based on the desirability of the result

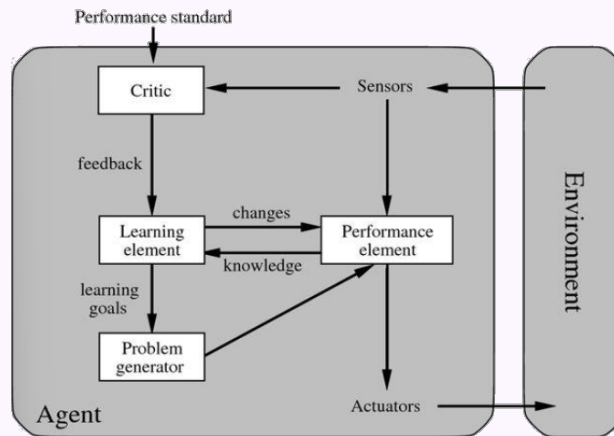
- Goals provide binary distinction, while utility functions provide a continuous measure of desirability
- Can handle selection between two conflicting goals - "Speed or safety"



Learning Agent

Employs additional learning element to gradually improve and become more knowledgeable over time about an environment

- Can learn from past experiences
- Is more robust towards unknown environments
- A learning agent has four conceptual components:
 1. **Performance Element:** Chooses external actions.
 2. **Learning Element:** Improves the performance element based on performance feedback
 3. **Critic:** Evaluates performance against a fixed performance standard and provides feedback to the learning element
 4. **Problem Generator:** Proposes actions that will lead to new and informative experiences (exploration)



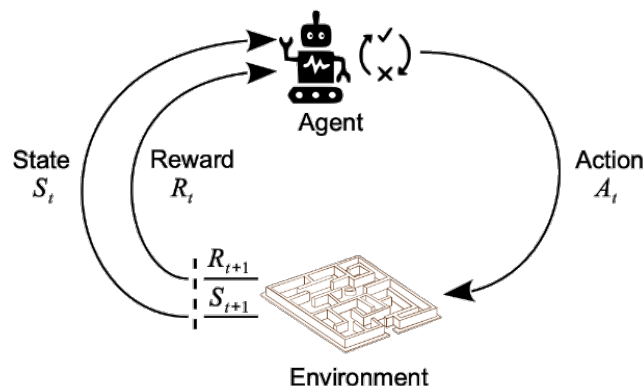
2.2 Creating Intelligent Agents

2.2.1 Search Algorithms

Define "finding a good action" as a search problem and use search algorithm to solve it. Classical search algorithms can be formulated as tree or graph search. Common options include BFS, DFS, UCS, and A*, and differ in their selection strategies and performance.

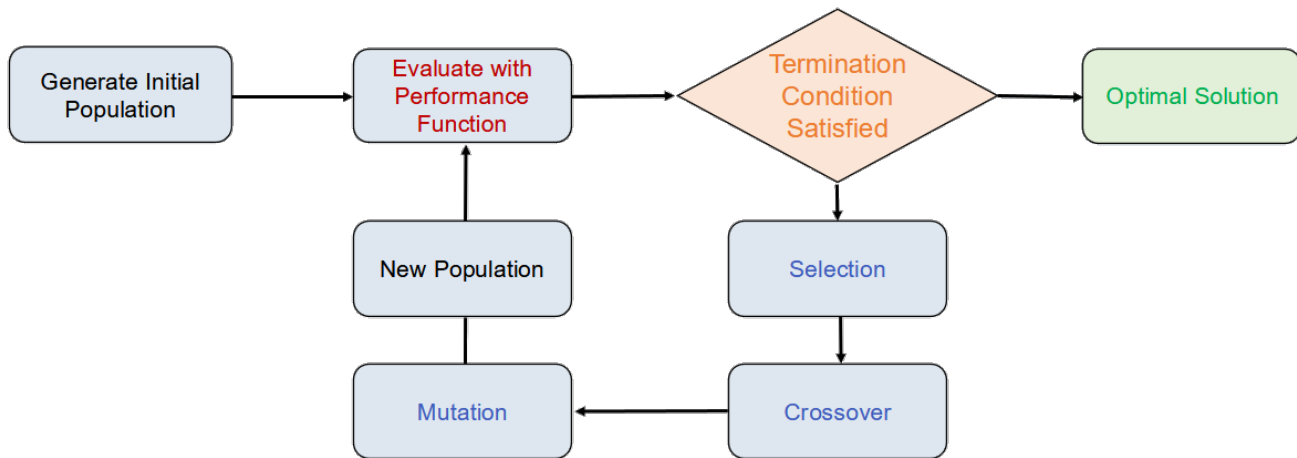
2.2.2 Reinforcement Learning

First defined in psychology, Reinforcement Learning essentially is a process of trial and error. Try something out, have a reaction to it, learn whether it was good or bad. Reactions or actions are based on our observations or experiences.



2.2.3 Genetic Algorithms

Inspired by Darwins "Theory of natural selection", this model build multiple different agents and evaluates every one of them. The agents with the highest performance are selected and new models are built based on it. It can also combine the best features of multiple agents to create new agents, creating a sort of "evolutionary" process.



3 Uninformed and Informed Search

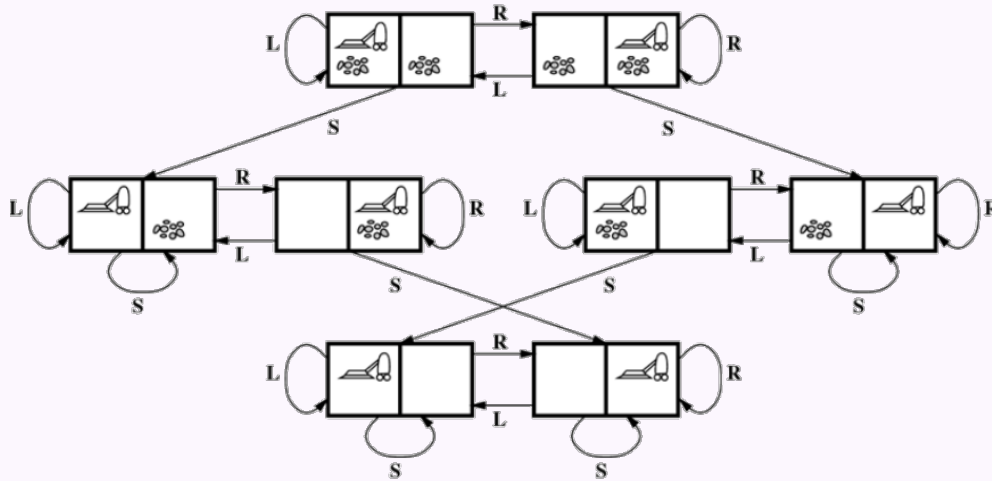
3.1 Problem Solving

3.1.1 Key Terminology

State Space / States

A state is a possible situation in our environment - The State Space is a set of all possible states, reachable from the initial state.

Often visualized as a **state-space graph** that displays all reachable states and its transitions.



State-Space Graph of a robot vacuum cleaning

Transition / Action

A Transition describes possible actions to take to get from one state to another. We only count direct transitions between two states (single actions)

Costs

Transitions often differ in different qualities. We add a "cost" to each action, so we can rate an algorithm on how cost effective it is.

Path

A sequence of states connected by a sequence of actions

Solution

A path that leads from the initial state to a goal state

Optimal Solution

The Solution with the minimal path cost

Problem Components:

1. State Space and Initial State:

All possible states and the initial environment as a state

2. Descriptions of Actions:

Function that maps a state to a set of possible actions in this state

3. Goal Test:

Typically a function to test if the current state fulfills the goal

4. Costs:

A cost function that maps actions to costs

An easy way it to add costs of all actions taken

Most problems can be defined as **planning problems**, in which we start from an initial state and transform it into a desired goal considering future actions and outcomes.

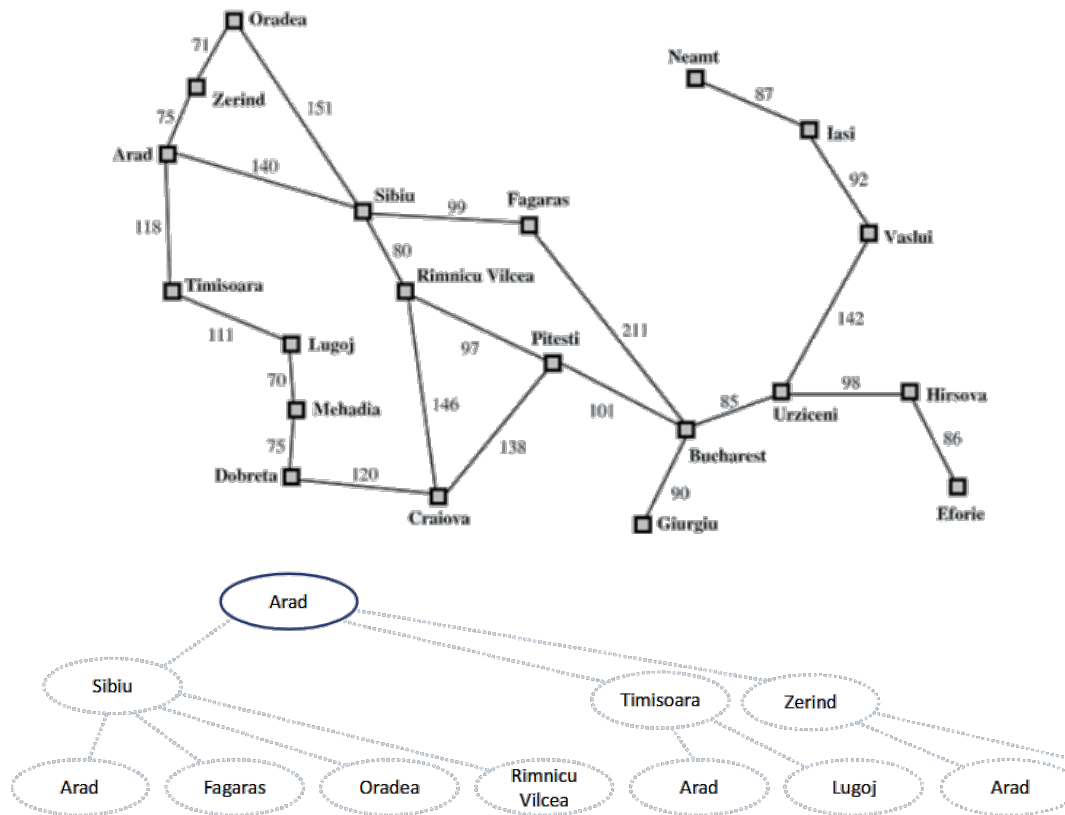
To solve these problems we usually utilize **search** algorithms to find a (optimal) solution in form of a sequence of actions.

3.1.2 Tree Search Algorithms

Hereby we treat the state-space graph as a tree, starting with the initial state. Using this approach we can define iterative or recursive algorithms to search for suitable paths.

Example Tree Search

```
1 Function tree_search(problem, strategy):
2   initialize search tree with initial state of problem;
3   While true do
4     If Node contains goal state then
5       return solution
6     Else If No suitable candidates for expansion then
7       return failure
8     Else
9       Expand node and add resulting nodes to the tree
```



Each Node in the search tree is an entire path in the state-space graph. We construct both on demand: only as much as we need (won't get all solutions)

Node

Describes a part of a tree. Includes **state**, **parent node**, **taken action**, **path cost** and **depth of the tree**

Fringe

Describes the set of nodes at the end of all visited paths

Depth

Number of levels in the search tree

3.2 Uninformed Search

Definition

The uninformed search strategy only has the problem definition, no additional information about "where" to look for solutions. It is also called **blind search**.

3.2.1 Uniform-Cost Search (UCS)

In UCS each node is associated with a fixed cost, which accumulate over the path. UCS uses the lowest cumulative cost to find a path.

UCS requires non-negative step costs for optimality. It is complete if step cost $\epsilon > 0$.

Complexity: $O(b^{1+\lceil C^*/\epsilon \rceil})$ with b being the branching factor (average number of children per node), C^* being the optimal solution cost, and ϵ being the minimal cost for a step

3.2.2 Breadth-First Search

BFS is a special case of UCS, when all costs are equal. It starts at the root and goes through each node of a level before progressing to the next level. BFS stops as soon as it finds a solution, while UCS searches for the 'best' solution by lower cost.

Complexity: $O(b^d)$ with b being the branching factor and d being the depth of the search tree

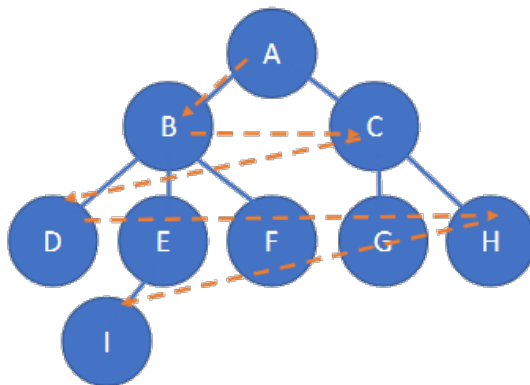
3.2.3 Depth-First Search (DFS)

Depth-First Search starts at the root and continues on one branch until it finds a solution or failure. In case of failure it backtracks to the current parent node and continues on the next branch.

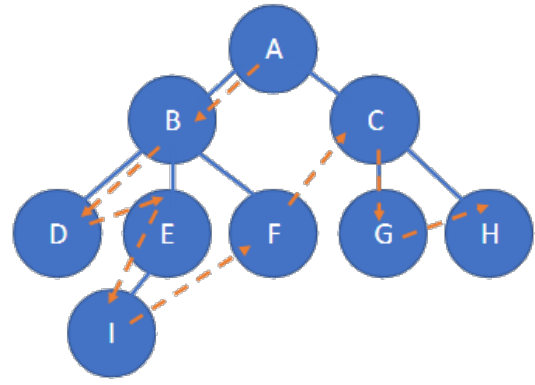
Unlike UCS and BFS DFS is not complete; It fails if the search space is of infinite depth or has loops.

It's also not optimal; More costly solutions may be found before less costly ones.

Complexity: $O(b^m)$ with b being the branching factor and m being the maximum depth of the search tree



BFS: A,B,C,D,E,F,G,H,I



DFS: A,B,D,E,I,F,C,G,H

Criteria	BFS	DFS
Concept	Traversing tree level by level	Traversing tree sub-tree by sub-tree
Data Structure (Queue)	First In First Out (FIFO)	Last In First Out (LIFO)
Time Complexity	$O(b^d)$	$O(b^m)$
Backtracking	No	Yes
Memory	Requires more memory	Less nodes are stored normally (less memory)
Optimality	Only when all costs are identical	Not without modification
Speed	In most cases slower compared to DFS	In most cases faster compared to BFS
When to use	If the target is relatively close to the root node	If the goal state is relatively deep in the tree

Comparison between BFS and DFS algorithms.

3.2.4 Depth-Limited Search (DLS)

DLS is a variant of DFS. Hereby the search is limited to a depth of l . This means that no infinite search can occur, the trade-off being that it might not find all solutions.

Example Depth-Limited Search

```
1 Function depth_limited_search(problem, limit):
2   | recursive_dls(make_node(initial_state(problem)), problem, limit)
3 Function recursive_dls(node, problem, limit):
4   cutoff_occured = false;
5   If goal_test(problem, state(node)) then
6     | return node;
7   Else If depth(node) >= limit then
8     | return cutoff;
9   Else
10    ForEach successor in expand(node, problem) do
11      result = recursive_dls(successor, problem, limit);
12      If result == cutoff then
13        | cutoff_occured = true;
14      Else If result != failure then
15        | return result;
16  If cutoff_occured then
17    | return cutoff;
18  Else
19    | return failure
```

This is generally neither complete nor optimal. It can be complete if the depth limit l is greater than or equal to the depth of the shallowest solution $d \Rightarrow l \geq d$. It is not optimal, as it may find a solution that is not the least costly.

Complexity: $O(b^l)$ with b being the branching factor and l being the maximum depth of the search tree

3.2.5 Iterative Deepening Search (IDS)

IDS is a variant of DLS. It works like DLS, but instead of a fixed depth, it iteratively increases the depth until a solution is found.

Example Iterative Deepening Search

```
1 Function iterative_deepening_search(problem):
2   For depth = 0 to  $\infty$  do
3     result = depth_limited_search(problem, depth);
4     If result != cutoff then
5       | return result
```

This change makes it complete and optimal.

Complexity: $O(b^d)$ with b being the branching factor and d being the depth of the search tree. This makes its time complexity equal to BFS. However, the space complexity of IDS is $O(bd)$, which is much better than BFS's $O(b^l)$.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes*	Yes*	No	Yes, if $l \geq d$	Yes
Time	b^d	$b^{1+\lceil C*/\epsilon \rceil}$	b^m	b^l	b^d
Space	b^d	$b^{1+\lceil C*/\epsilon \rceil}$	bm	b^l	bd
Optimal?	No*	Yes*	No	No	Yes*

Comparison of search strategies.

3.3 Heuristics h

Denotes a "rule of thumb", a rule that may be helpful in solving a problem.

In Tree-Search, a heuristic is a function that estimates the remaining cost from the current node to the goal.

Can go wrong!

Admissible Heuristic

A heuristic is admissible if it never overestimates the actual cost to reach the goal.

$$h(n) \leq h^*(n)$$

if $h^*(n)$ is the true cost to reach the goal from node n

Consistent Heuristic

A heuristic is consistent if for every node n and successor n' generated by any action a :

$$h(n) \leq c(n, a, n') + h(n')$$

if $c(n, a, n')$ is the cost of the action a from n to n'

Thus, a heuristic is consistent if, when going from neighboring nodes, the heuristic difference / step cost never overestimates the actual cost.

Lemmas

- If a heuristic is consistent, it is also admissible.
- If $h(n)$ is consistent, then the values of $f(n)$ on any path are non-decreasing.

3.3.1 Relaxed Problems

A relaxed problem is a problem that has fewer constraints on the actions than the original problem.

The cost of the optimal solution to a relaxed problem is an admissible heuristic for the original problem.

Example: Relaxed Problem as Admissible Heuristic

Consider a grid-based pathfinding problem where certain movements are restricted due to obstacles. A relaxed version of this problem might allow movement through obstacles by ignoring some constraints. The cost of the optimal solution to this relaxed problem, which is typically lower or equal to the original problem's optimal cost, serves as an admissible heuristic for the original problem.

This means that looking for relaxed problems is a good way to find admissible heuristics.

3.3.2 Dominance

A heuristic h_2 dominates h_1 if $h_2(n) \geq h_1(n)$ for all nodes n . (Given that h_1 and h_2 are admissible)

This means that a dominant heuristic is always closer to the optimal heuristic h^* , which results in less expansion and thus more efficient search.

3.3.3 Combining Heuristics

If $h_1, h_2, \dots, h_m$ are admissible heuristics, then $h(n) = \max(h_1(n), h_2(n), \dots, h_m(n))$ is also admissible and dominates both h_1 and h_2 .

This is useful if we have multiple admissible non-dominated heuristics and want to combine them to get a better heuristic.

3.4 Informed Search

Definition

The informed search strategy has additional knowledge about "where" to look for solutions, usually in form of a heuristic function.

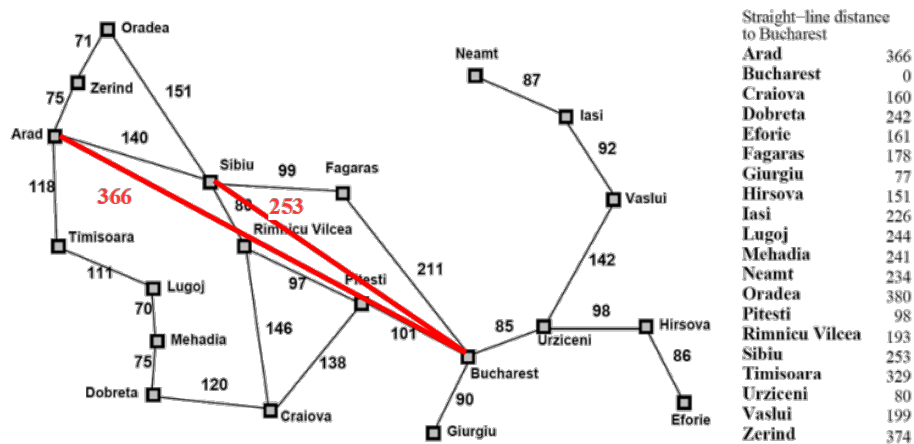
Some algorithms that utilize informed search are:

- Greedy Best-First Search
- A* Search
- Memory-Bounded Heuristic Search

3.4.1 Greedy Best-First Search

Greedy Best-First Search (GBFS) is an informed search algorithm that selects the next node to expand based on a heuristic that estimates the cost from the current node to the goal.

One example of a heuristic is the straight-line distance to the goal.



Example of GBFS with a straight-line distance heuristic.

Complexity: The time complexity of Greedy Best-First Search is $O(b^m)$, and its space complexity is also $O(b^m)$, where b is the branching factor and m is the maximum depth of the search tree. This is the same as DFS, however due to the heuristic it can be significantly faster.

GBFS is generally neither complete nor optimal in infinite spaces or without repeated state checking. In a finite space with repeated state checking, it can be complete, but it does not guarantee an optimal solution as it depends on the heuristic, which may be flawed.

- **Complete?** No*, can get stuck in loops.

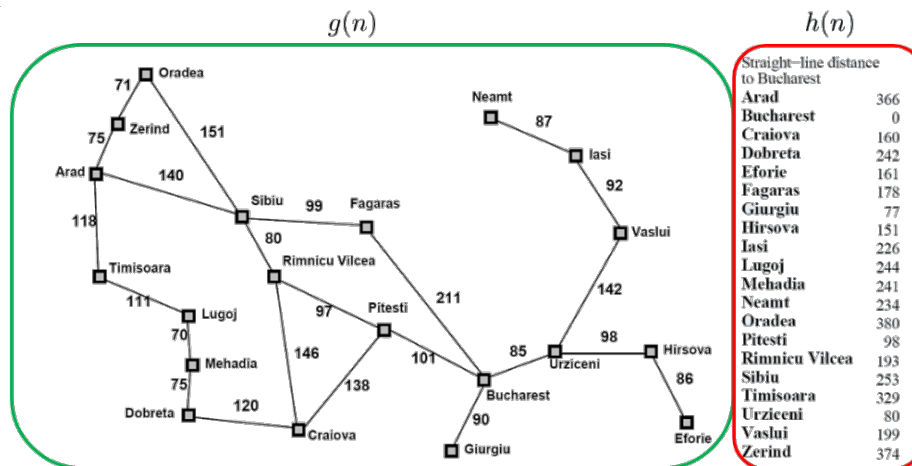
- **Optimal?** No, does not guarantee an optimal solution as it depends on the heuristic, which may be flawed.

3.4.2 A* Search

Definition

A* Search is an informed search algorithm that combines the strengths of Uniform-Cost Search and Greedy Best-First Search. It searches for the least-cost path to the goal by considering both the cost to reach the current node and an estimated cost to reach the goal.

A* Search tries to avoid paths that are already expensive. It evaluates the complete path cost and the remaining cost to the goal. Its cost function is defined as $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach the current node (root to n) and $h(n)$ (the heuristic) is the estimated cost to reach the goal (n to goal).



Example of A* Search with heuristic function.

Complexity: The time complexity of A* Search is $O(b^m)$, and its space complexity is also $O(b^m)$, where b is the branching factor and m is the maximum depth of the search tree.

Completeness and Optimality: A* Search is complete. Tree-Search A* is optimal with an admissible heuristic, assuming positive step costs. Graph-Search A* is optimal with a consistent heuristic.

3.4.3 Alternatives to A*

1. Iterative-Deepening A* (IDA*)

- Like iterative deepening, it explores nodes level by level, but uses A* to evaluate the nodes.
- Cutoff information is the f-cost ($g + h$) instead of the depth.

2. Recursive Best-First Search (RBFS)

- Recursive algorithm that mimics best-first search with linear space
- Keeps track of f-value of best alternative path
- Path available from any ancestor of the current node and heuristic evaluations are updated with results of successors

3. (Simple) Memory-Bounded A* ((S)MA*)

- Drops the worst leaf node when memory is full
- The dropped node's value is replaced by its parent's value
- May be re-searched later

3.4.4 Graph Search

When traversing a problem, loops can occur. Failure to detect them can turn linear search into exponential search. To avoid this, we can use **graph search**. Hereby we only expand nodes that have not been visited yet.

For example: The graph search version of UCS is **Dijkstra's algorithm**.

4 Local and Adversarial Search

(Un-)Informed Search shows some limitations. Memory and time requirements are often too high, especially for large state spaces.

4.1 Optimization Problems

An optimization problem is a problem where every state can be a solution (to different degrees) but the target is to find the state that optimizes (min, max,...) the solution according to an objective function.

Often, this means that only the final state's objective value matters, not the path taken to get there, thus it is not retained.

For example: Darwinian evolution could be seen as an optimization problem.

Objective (Evaluation) Function

An objective function shows how good a state is, also in comparison to other states. Its value is minimized or maximized depending on the problem.

4.1.1 Terminology

Convergence

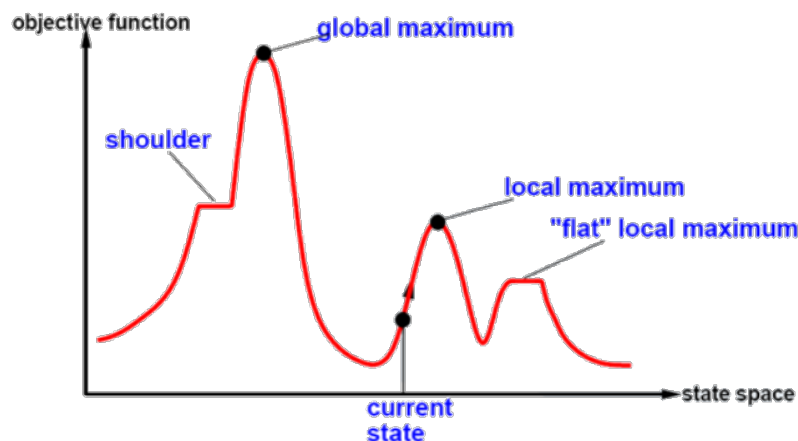
Describes a sequence of (function) values that approach a limit / value more and more.

Global Optimum

The **extremum** of an objective function over the **entire** input search space

Local Optimum

The **extremum** of an objective function over a **subset** of the input search space



4.2 Local Search

Local Search algorithms traverse only a single state rather than saving multiple paths. It modifies its state iteratively, trying to improve a specific criteria.

Optimization problems often times do not care about the path taken, but only to fulfill the goal constraint.

Advantages

- Uses little and constant memory
- Finds reasonable solution in large state spaces

Disadvantages

- No guarantee for completeness or optimality

Basic Idea (Travelling Salesman Problem):

- Start with a complete but likely suboptimal tour
- Modify the tour (e.g. pairwise swap) to improve the objective function
- Repeat until the tour is sufficiently good
- This approach often gets very good results quickly

4.2.1 Hill Climbing

Basic Idea:

- Expand the current state
- Move to the one with the highest value
- Repeat until a maximum is reached

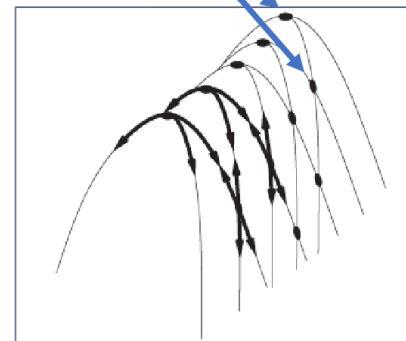
Hill Climbing

```
1 Function hill_climbing(problem):  
2   current = make_node(initial_state(problem));  
3   neighbor;  
4   While true do  
5     neighbor = highest_valued_successor(current);  
6     If value(neighbor) ≤ value(current) then  
7       return state(current);  
8     current = neighbor;
```

Ridge Problem:

Most Local Search algorithms are implemented by expanding their neighbors and then selecting the one that increases the objective function the most. Often times the problem space is not as simple as that there's only two neighbors, resulting in a more "3D" or higher search space. This can cause an issue, when all neighbors are worse than the current state, but the search space has an uphill.

States / steps (discrete)

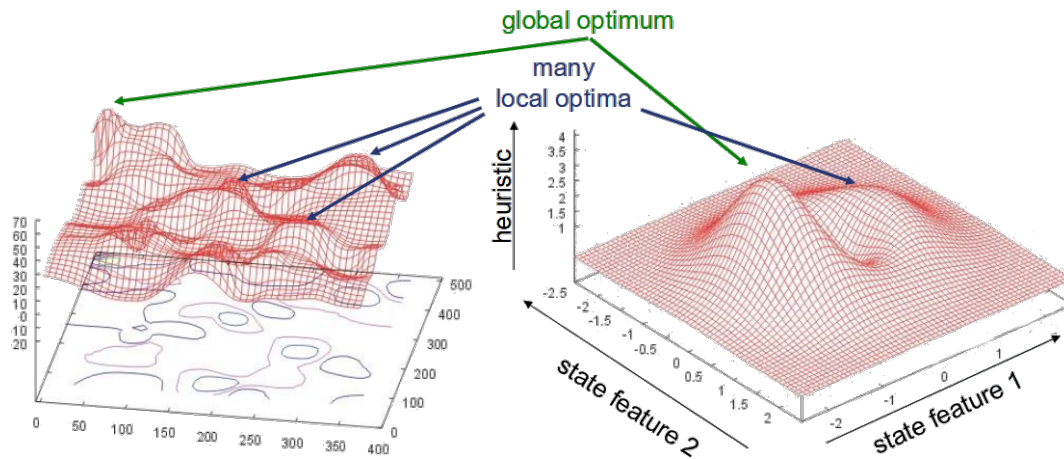


Problem - Local Optima:

- Algorithm will stop at the nearest local optima
- This might be very far from the global optimum (plateaus, ridges, shoulders...)
- Solution: Random Restart
 - Random initial positions result in different local optima
 - → Iterate over multiple local optima and select the best one
- Alternative Solution: Stochastic Hill Climbing
 - Select successor randomly with a higher chance for better successors

4.2.2 Gradient Descent

As mentioned before, search spaces often are not only 2-dimensional, as we might need to consider multiple features. This, of course, makes it much harder to find an optimal solution with higher dimensional search spaces.



Before our objective function only had a single input feature, now we also have to consider more.

Gradient

The **derivative** of a function that has more than one input variable. In mathematics it would be known as the **slope** of a function, which measures the change in all weights with regard to the change in error. The gradient is a vector that points in the direction of the steepest ascent of a function. The negative gradient points in the direction of the steepest descent.

Gradient Descent

The **gradient descent** is an optimization algorithm. It can be considered as Hill-Climbing in continuous state space.

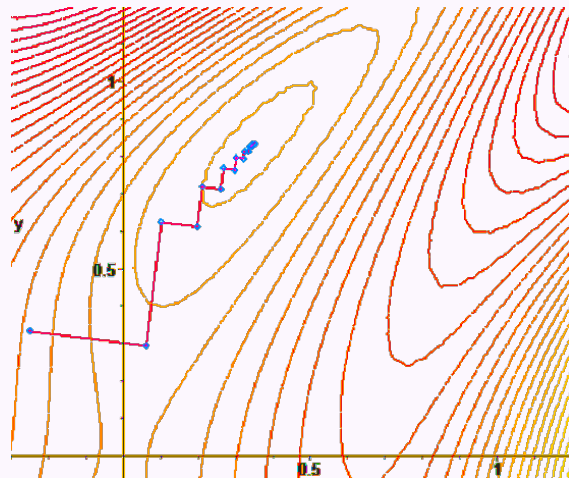
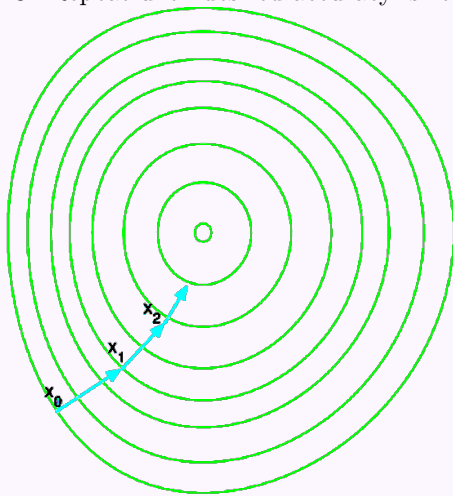
Gradient Descent: Working Principle

Gradient

$$\nabla J(\theta) = \frac{\partial J(\theta)}{\partial \theta} = \left[\frac{\partial J(\theta)}{\partial \theta_0}, \dots, \frac{\partial J(\theta)}{\partial \theta_n} \right] = \sum_{i=0}^n \frac{\partial J(\theta)}{\partial \theta_i}$$

We now want to minimize θ over the continuous variables. These can be any parameters, e.g. weights or biases. $x_1, x_2, \dots, x_n$.

1. Compute gradient: $\frac{\partial}{\partial x_i} J(x_1, x_2, \dots, x_n) \quad \forall i \in n$
 2. Take a step downhill in the direction of the gradient: $x_i' = x_i - \alpha \frac{\partial}{\partial x_i} J(x_1, x_2, \dots, x_n)$.
- We subtract the gradient because we want to minimize the function, not maximize it.
3. Repeat until desired accuracy is reached



Learning Rate α

"The size of the step taken in the gradient descent". The **learning rate α** is a hyperparameter, controlling how quickly the model adapts to the problem.

Finding the right learning rate is a very important task. It can be done by trial and error, or by using a learning rate scheduler.

In general:

- **Smaller learning rate:**
 - Smaller changes $\rightarrow$ requires more training epochs
- **Larger learning rate:**
 - Larger changes $\rightarrow$ requires fewer training epochs
 - Can overshoot minima, oscillate around minima, or even diverge

Determining the gradient can be difficult.

- Derive formula using multivariate calculus
- Ask mathematician or domain expert

- Literature search
- Automatic differentiation

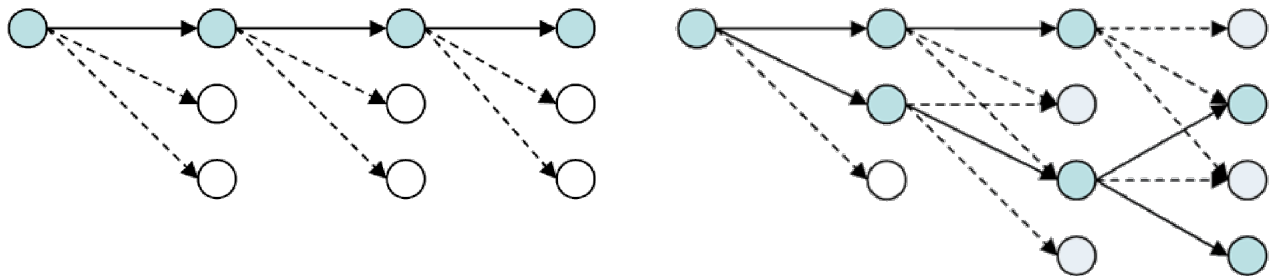
In practice, we usually use **backpropagation** to compute the gradient of a neural network. More on that in Section 11.

Gradient Descent in general works well for "smooth" spaces; poorly for "rough" spaces. Ideas on how to improve upon this are talked outside of the scope of this course. If you're interested check out the courses **Data Mining and Machine Learning** and **Deep Learning: Architectures and Methods**.

4.2.3 Beam Search

Basic Idea:

- Keep track of k states (beam size) rather than just one
1. Start with k randomly generated states
 2. At each iteration, all successors of all k states are generated
 3. Select the k best successors from complete list
 4. Repeat



4.2.4 Simulated Annealing

Basic Idea:

- Use hill-climbing, but occasionally take a step into a direction that does not show improvement
- Reduce the probability of a down-hill step as the number of iterations grows
- Allows some "bad moves" to escape local optima

Simulated Annealing Algorithm

```
1 // schedule maps time t to temperature T
2 Function simulated_annealing(problem, schedule):
3   current = make_node(initial_state(problem));
4   temperature;
5   next;
6   For t = 1 to ∞ do
7     temperature = schedule[t];
8     next = select_random_successor(current);
9     ΔE = value(next) - value(current);
10    If ΔE > 0 then
11      current = next;
12    // Still accepts worse solution with a probability of  $e^{\Delta E / \text{temperature}}$ 
13    Else If random_number(0,1) <  $e^{\Delta E / \text{temperature}}$  then
14      current = next
```

Temperature

Temperature is a hyperparameter, controlling how frequently we accept worse solutions to escape local optima. Temperature usually decays exponentially.



Simulated Annealing converges to a global optimum **if** the temperature is lowered slowly enough. This is not a strong claim as even random guessing could eventually yield the global optimum.

As such, simulated annealing can take a very long time.

4.3 Adversarial Search

Adversarial Search assumes an "adversary", who acts against the agent. The goal of adversarial search is to plan ahead, while taking the adversary's actions into account.

Adversarial search is often used to model games as search problems. Each player has to consider other players' actions and their effect on the game state.

Adversarial search is often time constrained, so it is unlikely to find an optimal solution.

4.3.1 Games

Zero-Sum Game

Describes a game where one player's gain is another player's loss. The sum of all players' utility values is always zero. If one player wins, the other player loses. If one player draws, the other player draws as well.

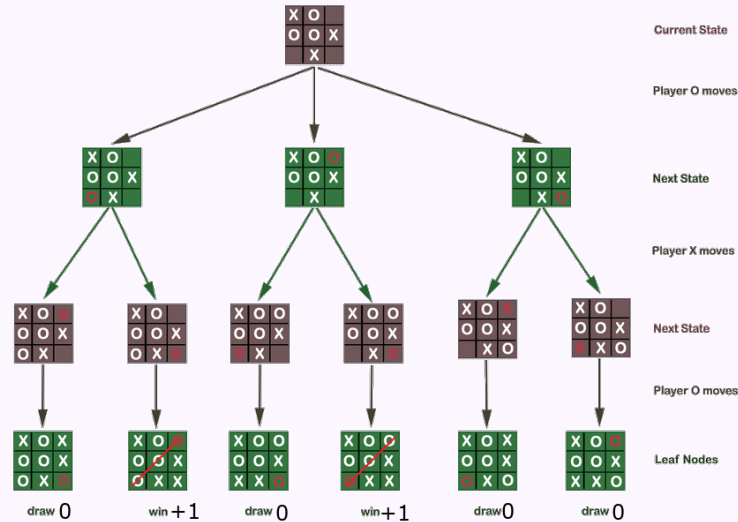
A game can be defined as a search problem:

1. **Initial State:** Game set-up
2. **Player(s):** Specifies which players turn it is

3. **Action(s)**: Returns all possible moves in state s
4. **Result(s, a)**: Specifies the state s' after action a in state s is taken
5. **Terminal(s)**: Tests if state s fulfills the goal/terminal constraints
6. **Utility(s, p)**: Returns a numeric value for a terminal state s from the perspective of player p

Game Trees

Game Trees are used to represent the possible states of a game. Hereby each level corresponds to a player. The **root node** is the initial state for the current problem. If the game has just begun, the root node would be the "empty" initial state. **Leaf Nodes** are always terminal states. Every **terminal node** has a utility value corresponding to the outcome of the game (e.g. +1 for a win, 0 for a draw, -1 for a loss).



4.3.2 Games vs. Search Problems

- "Unpredictable" opponent
 - Specify a move for every possible reply
 - Different goals for every agent
- Time limits
 - Likely not enough time to find a goal state
 - Needs approximation
- Classical games assume
 - Deterministic, turn-based, two-player, zero-sum, perfect information
- Real problems are
 - Stochastic, parallel, multi-agent, utility based, partial information

4.3.3 Minimax Algorithm

Basic Idea:

- Build a game tree where nodes represent the states of the game and edges represent the possible moves.
- The players:
 - **MIN**: Decreases the chances of **MAX** winning (Opponent)

- **MAX**: Increases his own chances of winning (Agent)
- Players take alternating turns following their respective strategies.
- Choose move to position with the highest **minimax value**
- Assume opponent to play the best response to their own action

Minimax Algorithm

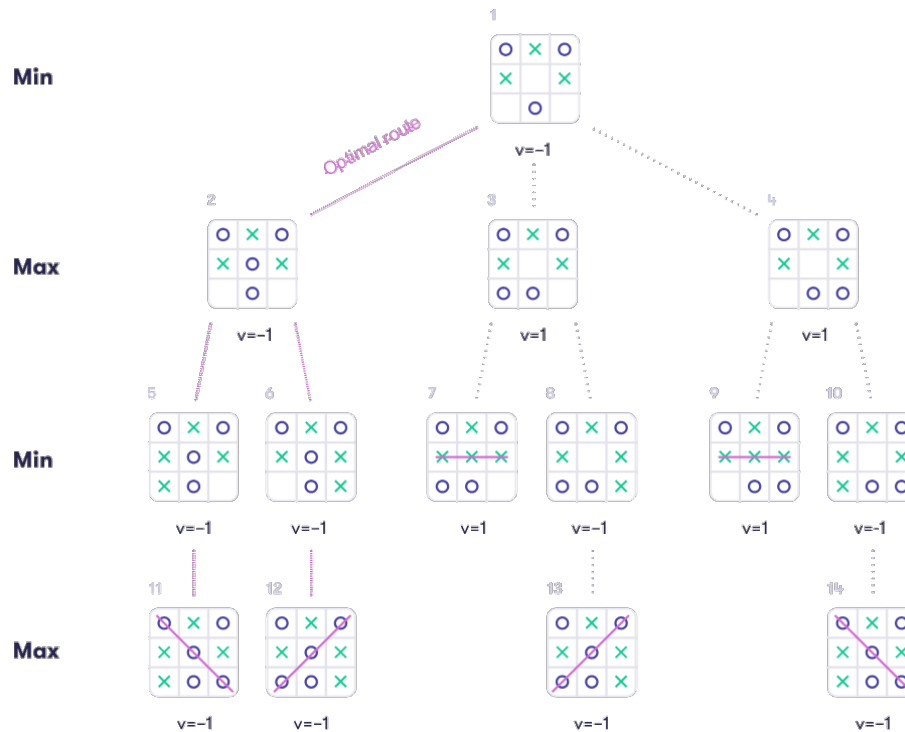
```

1 Function minimax_decision(state):
2   v = maxval(state);
3   // Returns the action that leads to state with utility value v
4   return get_action(v, get_successors(state));
5 Function maxval (state):
6   If is_terminal(state) then
7     return utility(state);
8   v =  $-\infty$ ;
9   For s in get_successors(state) do
10    v = max(v, minval(s));
11  return v;
12 Function minval (state):
13  If is_terminal(state) then
14    return utility(state);
15  v =  $\infty$ ;
16  For s in get_successors(state) do
17    v = min(v, maxval(s));
18  return v;

```

Characteristics

- **Optimality**: Yes, assuming an optimal opponent
- **Completeness**: Yes, if the tree is finite (e.g. no infinite game loops)
- **Time Complexity**: $O(b^m)$
- **Space Complexity**: $O(b \cdot m)$
- (b is the branching factor, m is the maximum depth)



Problem:

Many games have too many possible moves and go on for too long which causes the time complexity to grow exponentially. Go for example has a branching factor of about 250 and can go on for 150+ turns, which would result in approximately 5×10^{359} iterations in the worst case.

4.3.4 Alpha-Beta Pruning

A modified, optimized version of **Minimax Algorithm**. It uses **pruning** to reduce the amount of exploration without compromising the correctness of minimax.

Alpha-Beta Pruning is based on two parameters

- **Alpha:** The best (highest-valued) choice found so far at any point along the path of the Maximizer to the root. Initial Value: $-\infty$
- **Beta:** The best (lowest-valued) choice found so far at any point along the path of the Minimizer to the root. Initial Value: $+\infty$

Basic Idea:

Remove all nodes which are not affecting the final decision, but slow down the algorithm.

Alpha-Beta Pruning Algorithm

```
1 Function alpha_beta(state):
2    $\alpha = -\infty$ ;
3    $\beta = +\infty$ ;
4    $v = \text{maximizer}(\textit{state}, \alpha, \beta)$ ;
5   return  $v$ ;
6 Function maximizer (state,  $\alpha$ ,  $\beta$ ):
7   If is_terminal(state) then
8     return utility(state);
9    $v = -\infty$ ;
10  For  $s$  in get_successors(state) do
11     $\text{eval} = \text{minimizer}(s, \alpha, \beta)$ ;
12     $v = \max(v, \text{eval})$ ;
13     $\alpha = \max(\alpha, v)$ ;
14    If  $\beta \leq \alpha$  then
15      break;
16  return  $v$ ;
17 Function minimizer (state,  $\alpha$ ,  $\beta$ ):
18  If is_terminal(state) then
19    return utility(state);
20   $v = +\infty$ ;
21  For  $s$  in get_successors(state) do
22     $\text{eval} = \text{maximizer}(s, \alpha, \beta)$ ;
23     $v = \min(v, \text{eval})$ ;
24     $\beta = \min(\beta, v)$ ;
25    If  $\beta \leq \alpha$  then
26      break;
27  return  $v$ ;
```

Differences to Minimax:

- Max Player will only update alpha
- Min Player will only update beta
- While backtracking, the node values will be passed to upper nodes instead of alpha and beta
- Alpha and beta will only be passed to child nodes

Problems:

- Needs a fast evaluation function
- Games with large branching factors (e.g. Go) exploration with alpha-beta pruning is very slow

5 Constraint Satisfaction Problems

Constraint Satisfaction Problem

Constraint Satisfaction is a problem formalism where a problem is solved when its solution satisfies certain constraints or rules of the problem.

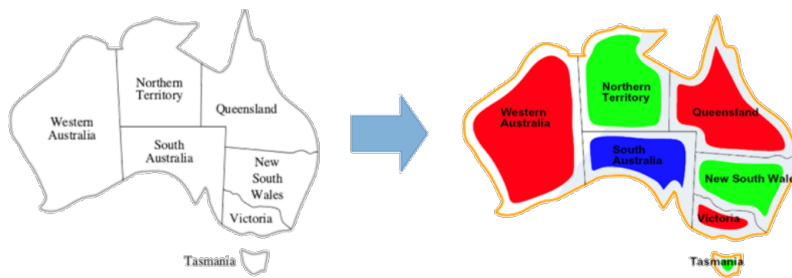
Components:

- A state, defined by **variables** X_i with **d** values from **domain** D_i
- A goal test, defined as a set of **constraints** C specifying allowable combinations of values for subsets of variables.

Solving Constraint Satisfaction Problems:

- A state space
- Notion of the solution

Example of a Constraint Satisfaction Problem



Problem: Assign each territory a colour such that no two adjacent territories have the same colour.

Variables: $X = \{WA, NT, Q, NSW, V, SA, T\}$

Domain of Variables: $D = \{red, green, blue\}$

Constraints: $C = \{SA \neq WA, SA \neq NT, SA \neq Q, \dots\}$

5.1 Assignment of Values to Variables

A state in state-space is not a "blackbox" as in standard search, but defined by assigning values to some or all variables.

$$X_1 = v_1, X_2 = v_2, \dots, X_d = v_d$$

The assignment of these values can be done in different ways:

1. **Consistent/Legal Assignment:** An assignment is consistent if it satisfies all constraints or rules.
2. **Complete Assignment:** An assignment is complete if every variable is assigned a value, and the solution to the CSP remains consistent.
3. **Partial Assignment:** An assignment is partial if some variables are not assigned values.

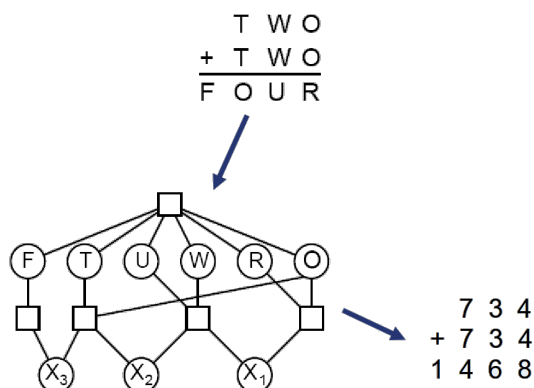
5.2 Constraint Graphs

Constraint Graphs are often constructed because abstraction of the problem makes it easier to solve and understand.

A constraint graph is usually denoted with

- Every variable is represented by a node
- Every edge indicates a constraint between two variables

Example of a Constraint Graph



Problem: Assign unique value to the variables of each letter, so that resulting equation is true.

Variables: $X = \{T, W, O, F, U, R\}$

Domain of Variables: $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Constraints: $C = \{T \neq W \neq O \neq F \neq U \neq R\}$

$\cup \text{int}("T" + "W" + "O") + \text{int}("T" + "W" + "O") = \text{int}("F" + "O" + "U" + "R")$

Here the connected nodes are involved in (in-)equations:

$$\begin{aligned} 2 \cdot O &= 10 \cdot X_1 + R \\ 2 \cdot W + X_1 &= 10 \cdot X_2 + U \\ 2 \cdot T + X_2 &= 10 \cdot X_3 + O \\ F &= X_3 \\ T \neq W \neq O &\neq F \neq U \neq R \end{aligned}$$

5.3 Types of Constraints

- **Unary constraints:** Involve a single variable (e.g. South Australia $\neq$ green)
- **Binary constraints:** Involve two variables (e.g. South Australia $\neq$ Wester Australia)
- **Higher-order constraints:** Involve more than two variables (e.g. $2 \cdot W + X_1 = 10 \cdot X_2 + U$)

Preferences / Soft constraints:

- Not binding, but should be considered during search
→ Constrained optimization problems
- e.g. Red is better than green

5.4 Solving CSPs: Search

Basic Idea:

1. Successively assign values to variable
2. Check constraints
3. If constraint is violated $\rightarrow$ backtrack
4. Repeat until all variables have assigned values that satisfy constraints

To do this we map CSPs into search problems:

- Nodes = assignments of values to a subset of the variables
- Neighbors of a node = nodes in which values are assigned to one additional variable
- Start node = empty assignment
- Goal node = a node which assigns a value to each variable and satisfies all constraints

5.4.1 Naive Search

Naive Search is practically a **brute-force** method. It systematically explores all possible assignments of v values to n variables. This, of course, is incredibly inefficient and results in exponential time complexity. The number of leaves in the search tree grows with $n!v^n$ if you consider all possible orderings.

5.4.2 Backtracking Search

Basic Idea: As assignments are commutative ($[WA = \text{red then } NT = \text{green}] = [Nt = \text{green then } WA = \text{red}]$) we can reduce the number of leaves in the search tree by only considering nodes that have not been visited before.

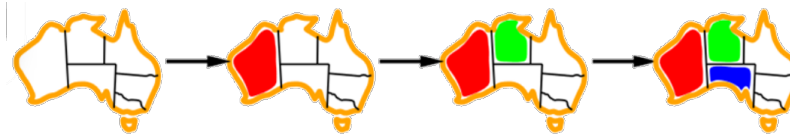
Backtracking Algorithm

```
1 Function backtrack_search(csp):
2   return recursive_backtrack(csp, {});
3 Function recursive_backtrack(csp, assignment):
4   If is_complete(assignment) then
5     return assignment;
6   var = get_unassigned_variable(get_variables(csp), assignment, csp);
7   ForEach value in order_domain_values(var, assignment, csp) do
8     If value is consistent with assignment given constraints(csp) then
9       assignment.add({var = value});
10      result = recursive_backtrack(csp, assignment);
11      If result != failure then
12        return result;
13      assignment.remove({var = value});
14   return failure;
```

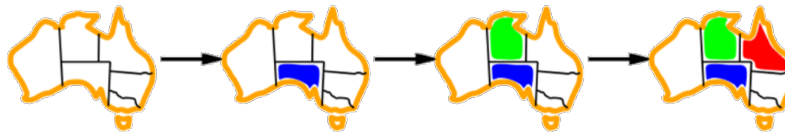
This is still not ideal as in the worst case the complexity is still exponential. This can be improved by including heuristics.

5.5 Heuristics for CSPs

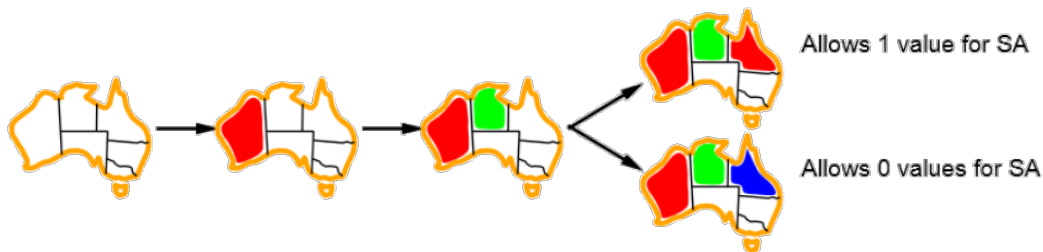
- **Domain-Specific Heuristics:** Depend on the particular characteristics of the problem.
- **General-Purpose Heuristics:** Can work on any CSP.
 - **Minimum Remaining Value:** Choose variable with fewest consistent values



- **Degree Heuristic:** Choose variable with the most constraints on remaining variables



- **Least Constraining Value Heuristic:** Given a variable, choose the value that rules out the fewest values in the remaining variables



A common strategy is MRV, Degree as a tie-breaker, and Least Constraining Value for value ordering. The effectiveness of these heuristics is problem-dependent.

5.6 Constraint Propagation

Node Consistency

A variable (node) is consistent if the possible values of this variable are conform to all unary constraints.

Local Consistency

Local consistency is a generalization of node consistency. A variable is locally consistent if it is consistent with all its neighbors. This is done by iteratively enforcing the constraint corresponding to the edges.

Arc

A constraint involving two variables is called an **arc** or binary constraint.

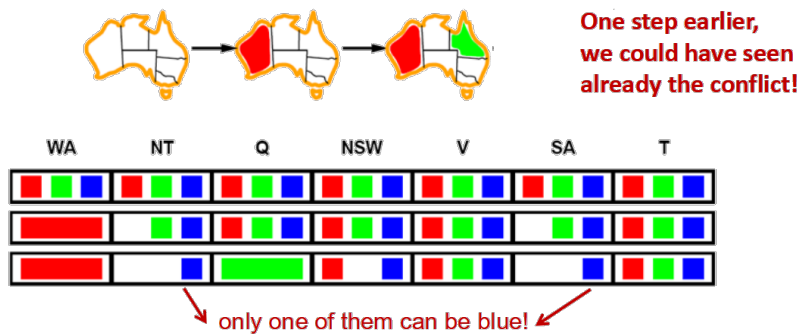
Arc Consistency

An arc is consistent if for each value of X in the domain of X there exists a value Y in the domain of Y such that the constraint $\text{arc}(X, Y)$ is satisfied.

$$\forall X \in \text{dom}(X), \exists Y \in \text{dom}(Y) : \text{arc}(X, Y) \text{ is satisfied}$$

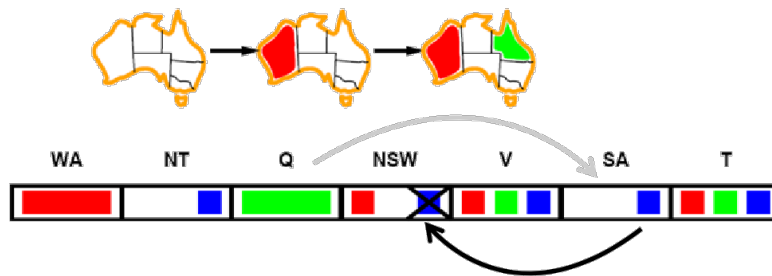
5.6.1 Forward Checking

Basic Idea: Keep track of remaining legal values for unassigned variables and terminate search, when any variable has no legal values left.

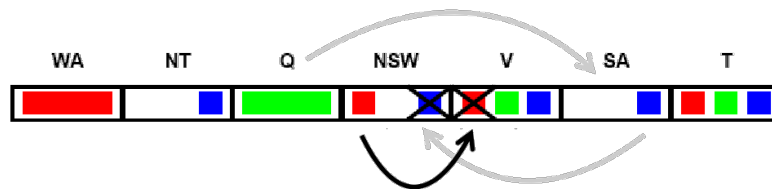


5.6.2 Maintaining Arc Consistency (MAC)

MAC enforces arc consistency after each assignment, usually by adding arcs affected by the assignment to a queue and checking them for consistency. This is done until the queue is empty.



If one variable (NSW) loses a value (blue), we need to recheck its neighbors as well because they might have lost a possible value.



AC-3 Algorithm

```
1 Function AC-3(csp):
2   queue = get_all_arcs(csp);
3   While queue is not empty do
4     (Xi, Xj) = remove_first(queue);
5     If remove_inconsistent_values(Xi, Xj) then
6       ForEach Xk in get_neighbors(Xi) do
7         queue.add((Xk, Xi));
8 Function remove_inconsistent_values(Xi, Xj):
9   removed = false;
10  ForEach x in get_domain(Xi) do
11    If no value y in get_domain(Xj) satisfies arc(Xi, Xj) then
12      get_domain(Xi).remove(x);
13      removed = true;
14  return removed;
```

5.6.3 Path Consistency

Arc consistency is often sufficient to:

- Solve the problem (all variable domains reduced to one value)
- Show that the problem cannot be solved (some domains empty)

but sometimes may not be enough, for example if there's always a consistent value in the neighboring region.

Path consistency tightens the binary constraint by considering triples of values.

A pair of variables (X_i, X_j) is path-consistent with X_m if for every assignment that satisfies the constraint on the arc (X_i, X_j) there is an assignment that satisfies the constraints on the arcs (X_i, X_m) and (X_j, X_m).

5.6.4 k-Consistency

k-Consistency is a generalization of path consistency. A CSP is k-consistent if every consistent assignment of k-1 variables can be extended to any k-th variable. It may lead to a faster solution but checking for k-consistency is computationally expensive with exponential time in the worst case.

In practice, arc consistency is most frequently used.

5.6.5 Constrain Propagation & Backtracking Search

Basic Idea: Each time a variable is assigned, a constraint propagation algorithm is run in order to reduce the number of choice points in the search. This can improve the speed of backtracking search further. This algorithm can be implemented using Forward Checking or AC-3.

5.7 Local Search for CSPs

Necessary Modifications for CSPs:

- work with complete states
- allow states with unsatisfied constraints
- operators reassign variable values

Min-Conflicts Heuristic:

- Randomly select a conflicted variable
- Choose the value that violates fewest constraints
- Hill climbing with $h(n) = \#$ of violated constraints

Performance:

- Can solve randomly generated CSPs with a high probability
- Except in a narrow range $R = \frac{\# \text{ of constraints}}{\# \text{ of variables}}$

5.8 Problem Decomposition

Assume search space for a constraint satisfaction with **n variables**, each of which can have **d values** $= O(d^n)$

Basic Idea: Decompose the problem into subproblems with **c variables** each

- Each problem has complexity $O(d^c)$
- There are n/c problems $\rightarrow$ total complexity $O(n/c \cdot d^c)$
- Unconditional independence is rare

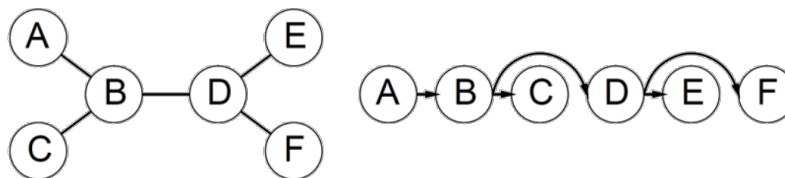
This can reduce the total complexity from exponential to linear, assuming c is constant.

5.9 Tree-Structured CSPs

A CSP is tree-structured if in the constraint graph any two variables are connected by a single path. Any tree structured CSP can be solved in $O(n \cdot d^2)$, which is linear in the number of variables (if d is fixed).

5.9.1 Linear Algorithm

1. Choose variable as root, order nodes so that parent always comes before its children (only one parent per node)
2. For $j = n$ downto 2
 - Make the arc(X_i, X_j) arc-consistent, calling `remove_inconsistent_value( $X_i, X_j$ )`
3. For $i = 1$ to n
 - Assign to X_i any value that is consistent with its parent



5.9.2 Nearly Tree-Structured Problems

Tree structured problems are rare.

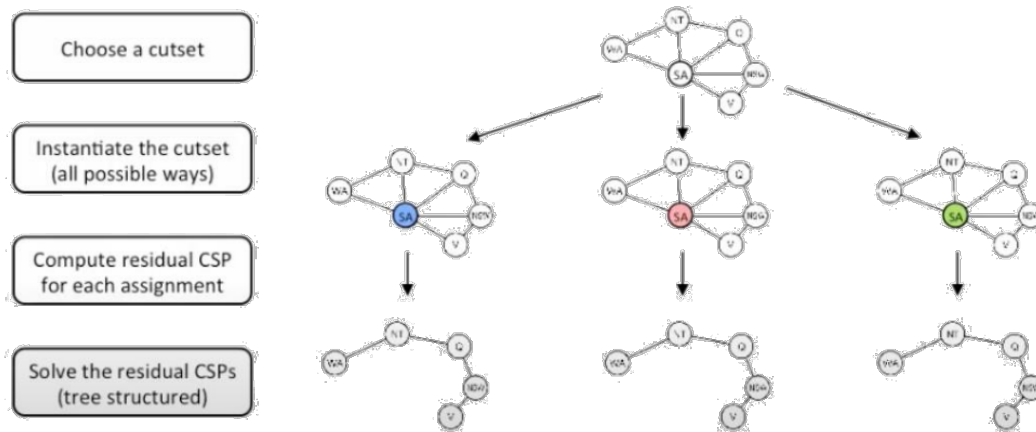
Approaches for making them tree-structured:

1. **Cutset Conditioning:**
 - Removing nodes so that the remaining nodes form a tree
2. **Collapsing nodes together:**

- Decompose the graph into a set of independent tree-shaped subproblems

Cutset Conditioning

1. Choose a subset S of the variables such that the constraint graph becomes a tree after removal of S (cycle cutset)
2. Choose a consistent assignment of variables for S
3. Remove from the remaining variables all values that are inconsistent with the variables of S
4. Solve the CSP problem with the remainign variables
5. If no solution: Choose different S in **2**



6 Logic & AI: Propositional Logic

Search Algorithms only evaluate states, but do not have an "understanding" of the environment. This does mean, that a goal might not even be able to exist logically, but the search algorithms will still search for it.

Propositional Logic aims to improve on that aspect.

6.1 Logic

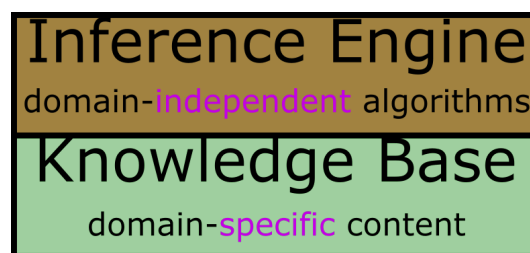
Logic is the key behind any formal knowledge. It allows to filter necessary information from a set of information and to draw conclusions. In AI, any representation of knowledge is based on logic.

Knowledge Base (KB)

A knowledge base is a collection of sentences in a formal language representing an agents information or beliefs about a domain. In real life, the agents knowledge base is often incomplete and may contain errors. The knowledge base is used to answer questions about the world and to make decisions.

Inference Engine

The inference engine is responsible for inferring new knowledge from the knowledge base. It is a central component of any knowledge-based agent.



Knowledge-Based Agents

A knowledge-based agent is a type of **intelligent agent** that uses a knowledge base and an inference engine to make decisions.

```
1 kb; // The knowledge base           t; // counter, indicating time
2 Function knowledge_based_agent(percept):
3   tell(kb, make_percept_sentence(percept,t));
4   action = ask(kb, make_action_query(t));
5   tell(kb, make_action_sentence(action,t)) t++;
6   return action
```

- Represent states, actions...
- Incorporate new percepts and update knowledge base
- Deduce properties of the world and make decisions / actions

6.2 Syntax

A sentence in propositional logic follows the **Backus-Naur Form (BNF)**:

Symbol: P, Q, R,... Descriptor of a sentence
Sentence: True | False | Symbol | Sentence / Statement that evaluates to a truth value
 $\neg(\text{Sentence})$ |
 $(\text{Sentence} \wedge \text{Sentence})$ |
 $(\text{Sentence} \vee \text{Sentence})$ |
 $(\text{Sentence} \Rightarrow \text{Sentence})$...

6.3 Semantics

Interpretation specifies which symbols are true and which are false. Given an interpretation it should be possible to evaluate a sentence.

A truth table defines semantics of operators:

a	b	$\neg a$	$a \wedge b$	$a \vee b$	$a \Rightarrow b$
false	false	true	false	false	true
false	true	true	false	true	true
true	false	false	false	true	false
true	true	false	true	true	true

6.4 Tautology

A tautology is a sentence that is true for all possible interpretations.

P	Q	$P \vee Q$	$\neg P \wedge \neg Q$	$P \vee \neg P$	$(P \vee Q) \vee (\neg P \wedge \neg Q)$
false	false	false	true	true	true
false	true	true	false	true	true
true	false	true	false	true	true
true	true	true	false	true	true

6.5 Logical Equivalence

Two sentences are **logically equivalent** if they have the same truth value for every setting of their propositional variables.

P	Q	$P \vee Q$	$\neg(\neg P \wedge \neg Q)$
false	false	false	false
false	true	true	true
true	false	true	true
true	true	true	true

Logical Law	Equivalence
Commutativity	$(a \vee b) \equiv (b \vee a)$ $(a \wedge b) \equiv (b \wedge a)$
Associativity	$((a \wedge b) \wedge c) \equiv (a \wedge (b \wedge c))$ $((a \vee b) \vee c) \equiv (a \vee (b \vee c))$
Double Negation Elimination	$\neg(\neg a) \equiv a$
Contraposition	$(a \Rightarrow b) \equiv (\neg b \Rightarrow \neg a)$
Implication Elimination	$(a \Rightarrow b) \equiv (\neg a \vee b)$
De Morgan's Laws	$\neg(a \wedge b) \equiv (\neg a \vee \neg b)$ $\neg(a \vee b) \equiv (\neg a \wedge \neg b)$
Distributivity	$(a \wedge (b \vee c)) \equiv ((a \wedge b) \vee (a \wedge c))$ $(a \vee (b \wedge c)) \equiv ((a \vee b) \wedge (a \vee c))$

6.6 Inference / Entailment

A sentence is **entailed** by the knowledge base if, for every setting of the propositional variables, for which knowledge base is true, the sentence is also true.

Assume 2 sentences, A and $A \Rightarrow B$:

A	B	Knowledge base
false	false	false
false	true	false
true	false	false
true	true	true

To find out whether a sentence A is entailed by knowledge base as simple algorithm can be used:

Basic Idea:

1. Go through all possible setting of the propositional variables
2. If knowledge base is true and A is false $\Rightarrow$ return false
3. Else $\Rightarrow$ return true

Problem: Not very efficient: The number of setting increases with $2^{\text{propositional variables}}$

6.6.1 Principle of Non-Contradiction

"A cannot be $\neg A$ "

Two contradictory statements cannot be true at the same time, as that permits **explosion**: Any statement can be inferred from a contradiction.

Example:

$\text{PetIsABird} \Rightarrow \text{PetCanFly}$

$\text{PetIsAPenguin} \Rightarrow \text{PetIsABird}$

$\text{PetIsAPenguin} \Rightarrow \neg(\text{PetCanFly})$

PetIsAPenguin

This would imply that a penguin can both fly and not fly. If you would work with this contradictory predicate you could infer any statement:

$\text{PetCanFly} \vee \text{MoonMadeOfCheese} \equiv \text{True}$

Here we say "Either the pet can fly or the moon is made of cheese". But since we also assume that the pet is a penguin, and therefore can't fly, we can conclude that the only remaining option to fulfill the statement is that the moon is made of cheese. This is obviously not true, but we could have inferred it from a contradiction.

6.7 Conjunctive Normal Form (CNF)

The CNF is a way to write any knowledge base as a single formula:

CNF Formula

$(\dots \vee \dots \vee \dots) \wedge (\dots \vee \dots \vee \dots) \wedge \dots$

- Can be a symbol x or $\neg(x)$ (**Literals**)
- Multiple facts in knowledge base are "AND"ed together

Example: $\text{RoommateWet} \Rightarrow (\text{RoommateWetOfRain} \vee \text{RoommateWetOfSprinklers})$

becomes

$(\neg(\text{RoommateWet}) \vee \text{RoommateWetOfRain} \vee \text{RoommateWetOfSprinklers})$

6.8 Modus Ponens

Modus Ponens allows to form new sentences from existing ones:

Assume two sentences, A and $A \Rightarrow B$: From this we can conclude the new sentence B .

6.8.1 Unit Resolution

Assume the sentences $l_1 \vee l_2 \vee \dots \vee l_k$ and $\neg(l_i)$.

From this we can conclude the new sentence: $l_1 \vee l_2 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k$

6.8.2 General Resolution

Assume two sentences $l_1 \vee l_2 \vee \dots \vee l_k$ and $m_1 \vee m_2 \vee \dots \vee m_n$ where for some i, j $l_i = \neg(m_j)$.

From this we can conclude the new sentence: $l_1 \vee l_2 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee m_2 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n$

The same literal may appear multiple times; these need to be removed.

6.9 Resolution

Satisfiable

There exists a model that makes the modified knowledge base true, i.e., the modified knowledge base is consistent.

To see if a knowledge base is satisfiable, one can use a resolution algorithm.

6.9.1 Resolution Algorithm

Basic Idea: CNF formula for modified knowledge base is satisfiable if and only if sentence A is **not entailed**. So to see if a sentence A is entailed we can simply add $\neg A$ to the knowledge base and see if it becomes inconsistent.

1. **Find** two clauses with complementary literals
2. **Apply** resolution
3. **Add** resulting clause (if not already there)
4. **Test**, if it results in the empty clause $\rightarrow$ formula is not satisfiable

Special Case: Horn Clauses

Horn Clauses

Horn clauses are a disjunction of literals with at most one positive (i.e. unnegated) literal. We can distinguish between Definite Clauses (exactly one positive literal), Goal Clauses (no positive literals) and Facts (exactly one positive literal and no negative literals).

- $X_1 \wedge X_2 \wedge \dots \wedge X_n \Rightarrow Y$ (Definite Clause)
- $X_1 \wedge X_2 \wedge \dots \wedge X_n \Rightarrow \text{False}$ (Goal Clause)
- $\text{True} \Rightarrow X_1$ (Fact)

To find out whether a literal X_j is entailed:

1. Start from known implications as far as possible
2. If X_j is reachable it is entailed

To increase efficiency of this approach we can maintain a count of how many implications are already known to reduce the necessary computations.

6.10 Limitations of Propositional Logic

- No notion of objects or relations:
 - Identifiers are merely suggestive, it does not necessarily mean that the implied objects and relations actually exist or are real.
 - To this end, every identifier might as well be a single letter $A, B \dots$

7 Logic & AI: First-Order Logic

As established before, Propositional Logic does not have any notion of relations or objects. This does of course limit the possible applications of Propositional Logic.

First-Order Logic (FOL) extends on propositional logic to allow for relations and objects.

Some commonly used terms in FOL and scientific papers:

Axioms

Basic facts about the domain, the "initial" knowledge base.

Theorems

Statements that are logically derived from axioms.

7.1 Elements in FOL

- **Objects:**
 - Represent entities in the real world - Can be named accordingly: Person1, John, Earth...
- **Relations:**
 - Represent relations between objects - Can be named accordingly: Has($\cdot$, $\cdot$), Is[Something]($\cdot$)...
 - Relations with only one object are called **Unary Relations or Properties**: Has[Something]($\cdot$), Is[Something]($\cdot$)...
- **Functions:**
 - A function maps a tuple of terms to a single term
 - E.g. Roommate($\cdot$) $\rightarrow$ Roommate(Person1) = Person2 $\wedge$ Roommate(Person2) = Person1
 - Can be used to encode integers and data structures:
 - * 0, succ(0) = 1, succ(1) = 2, succ(2) = 3...
 - * tree(value1, tree(value2,...), tree(value3,...))
 - Not specific to specific object: Roommate(Umbrella) is valid, but might be nonsensical

7.2 Quantifiers

Quantifiers can be used to refer to multiple objects at once.

Universal: For All $\forall$

Asserts that a statement is true for all objects.
 $\forall x : \text{Lion}(x) \Rightarrow \text{Cat}(x)$: All lions are cats

Existential: Exists $\exists$

Asserts that a statement is true for at least one object.
 $\exists x : \text{Cat}(x) \wedge \neg(\text{Lion}(x))$: At least one cat is not a lion

$\forall x : A$ is equivalent to $\neg(\exists x : \neg(A))$.

7.3 Substitution

Substitution: SUBST(·)

The SUBST method replaces one or more variables with something else in a sentence.

Example:

$\text{SUBST}(\{x / \text{John}\}, \text{IsHealthy}(x) \Rightarrow \neg(\text{HasACold}(x)))$
becomes
 $\text{IsHealthy}(\text{John}) \Rightarrow \neg(\text{HasACold}(\text{John}))$

7.3.1 Instantiating Quantifiers with SUBST

Universal $\forall$

Assuming a statement $\forall x : A$ we can obtain a new clause for **any** concrete **ground term g**:
 $\text{SUBST}(\{x / g\}, A)$

Example:

$\forall x \in \mathbb{R} : x > 0 \Rightarrow x^2 > 0$
Substituting any concrete ground term g , in this
case $g = 3$, yields:
 $\text{SUBST}(\{x / 3\}, x > 0 \Rightarrow x^2 > 0) =$
 $3 > 0 \Rightarrow 3^2 > 0$

Existential $\exists$

Asserts that a statement is true for at least one object.

Assuming a statement $\exists x : A$ we can obtain a new clause for **only one Skolem constant k**:
 $\text{SUBST}(\{x / k\}, A)$

Example:

$\exists x \in \mathbb{R} : x^2 = 9$
In this case we do not assign a concrete value, but
assign a "placeholder", the **Skolem constant k**,
that can later be instantiated:
 $\text{SUBST}(\{x / k\}, x^2 = 9) = k^2 = 9$

Skolem Constant

Important:

- k is a constant, that does not appear elsewhere in the knowledge base
- The result of $\text{SUBST}(\{x / k\}, A)$ is a clause that is **equisatisfiable** with the original clause. It is not generally logically equivalent, but it is satisfiable if and only if the original clause is satisfiable.
-

One must act carefully when Instantiating existentials **after** universals:

Assume the statement $\forall y \exists x : \text{Parent}(x, y)$: "Every person y has a parent x ".

Correct Instantiation:

1. Substitute x with a skolem function $f(y)$ to get a more specific sentence: $\forall y : \text{Parent}(f(y), y)$: "Every person y has a parent $f(y)$ ".
2. The statement then becomes $\exists x : \text{Parent}(f(y), y)$: "Every person y has a parent $f(y)$ ". This is correct, as the parent of y can be different for each y .
3. Assigning a concrete value to y later on does not disturb this logic, $f(y)$ will become a concrete value, but it will be different for each y .

Incorrect Instantiation:

1. Assign x a skolem constant k

2. The statement then becomes $\forall y : \text{Parent}(k, y)$: "Every person y has a parent k ". This would mean that every person y has **the same parent** k , which might not be correct.

7.4 Generalized Modus Ponens

7.4.1 Unification

Assume two sentences: $\forall x : \text{Loves}(\text{John}, x)$ "John loves everything" and $\forall y : (\text{Loves}(y, \text{Jane}) \Rightarrow \text{FeelsAppreciatedBy}(\text{Jane}, y))$ "Jane feels appreciated by everything that loves her".

We can now use substitution:

1. $\{x/\text{Jane}\} \rightarrow \text{Loves}(\text{John}, \text{Jane})$
2. $\{y/\text{John}\} \rightarrow \text{Loves}(\text{John}, \text{Jane}) \Rightarrow \text{FeelsAppreciatedBy}(\text{Jane}, \text{John})$
3. As 1 fulfills the condition of 2, we can infer that $\text{FeelsAppreciatedBy}(\text{Jane}, \text{John})$

7.5 First-Order Conjunctive Normal Form

1. Convert to **Negation Normal Form**: Negation symbols only occur immediately before predicate symbols
2. If variable names are used twice within scopes of different quantifiers, rename one of them such that the name is not used elsewhere
3. **Skolemize statements**:
 - (a) Move quantifiers out, so that we got $\forall x_1, x_2 \dots \exists y_1, y_2 \dots : A$. Quantifiers only occur in the prefix, not inside conjuncts.
 - (b) Replace existentially quantified variables with skolem constants or skolem functions.
 - (c) Discard universal quantifiers
4. **Convert** into clause set

Example:

Assume the sentence: $\forall x, y : \text{eats}(x, y) \wedge \neg(\text{killed}(x)) \Rightarrow \text{food}(y)$ "Anything anyone eats and is not killed is food"

1. Eliminate Implications: $\forall x, y : \neg(\text{eats}(x, y) \wedge \neg(\text{killed}(x))) \vee \text{food}(y)$
2. Move negations inwards (De Morgan): $\forall x, y : \neg(\text{eats}(x, y)) \vee \text{killed}(x) \vee \text{food}(y)$
3. Drop universal quantifiers: $\neg(\text{eats}(x, y)) \vee \text{killed}(x) \vee \text{food}(y)$

Inference in FOL is **not decidable** but **semidecidable**. If entailed, a proof may be found eventually, but if not entailed, the search may run forever. Satisfiability in FOL is **undecidable** in general.

7.6 Prolog

Prolog (Programming in Logic) is a FOL based logic programming language.

It is based on three basic components:

- **Facts**: Statements that are unconditionally true. E.g.:
 - `cat(john)` "John is a cat".
 - `parent(john, tom)` "John is the parent of Tom"
- **Rules**: Conditional statements that define relationships. E.g.:
 - `animal(X) :- cat(X)` "X is an animal *if* X is a cat"
- **Queries**: Questions asked to the program to derive answers. If multiple answers are possible, Prolog will return one per iteration in order. E.g.:

– ?- cat(tom) "Is Tom a cat?"

7.6.1 Atoms and Variables

Atom

A constant term that represents fixed values. It is denoted by lowercase letters.
Not every constant is an atom: Numbers, empty lists, some constructs (e.g. `parent(john, tom)`) are not atoms but constant.

Variable

A term that represents unknown or general values. It is denoted by uppercase letters or `_`.

7.6.2 Prolog as a Database

Prolog can be regarded as a standard relational database, that stores facts and rules, which can be accessed and manipulated using Prolog queries. One caveat is that prologs backtracking strategy to retrieve all queries can be inefficient.

7.7 Gödel's Incompleteness Theorem

Gödel's Incompleteness Theorem states that any consistent, effectively axiomatized formal system strong enough to express elementary arithmetic is incomplete. There are statements in the system that can neither be proved nor disproved within the system.

8 Uncertainty

Up until now we assumed that every statement is either true or false (or unknown) and that every action taken behaves exactly as expected.

In the real world, this is not always the case. Agents almost never have access to the entire state (incomplete information). This means that in many cases, **agents must deal with uncertainty**.

8.1 Probabilities

One way to deal with uncertainty is to use **probabilities**. We can assign a probability to every event, to summarize effects of uncertainty, due to:

- **Laziness:**
 - Agent is too lazy to consider all possible outcomes and possible events
- **Theoretical Ignorance:**
 - Some things are impossible to know (e.g. Weather cannot be definitely predicted)
- **Practical Ignorance:**
 - Some things might just not be known about an event or situation (e.g. traffic conditions)

Probabilities can be interpreted in several ways, including frequentist long-run frequencies and Bayesian subjective beliefs. Frequentist probabilities are based on the idea that the probability of an event is the limit of its relative frequency in a large number of trials. Bayesian probabilities, on the other hand, represent an agent's degree of belief in an event, which can be updated as new evidence is obtained. AI systems often use Bayesian probabilities to model uncertainty and make decisions based on incomplete information. Degree of belief and degree of truth are two different concepts that are often confused:

- **Degree of Belief:** There is a traffic jam in 10% of the cases
- **Degree of Truth:** The street is 10% jammed (blocked off a bit)

Probability theory is about **degree of belief** not **degree of truth**.

8.1.1 Basics

The state or **sample space** Ω can be seen as a set of all **samples** ω :

$$\begin{array}{c} \Omega \\ \omega \in \Omega \end{array}$$

The **probability space** or probability model is a sample space with an assignment of probabilities to all samples:

$$\forall \omega \in \Omega : \exists P(\omega) : \sum_{\omega \in \Omega} P(\omega) = 1$$

This is a simplified version of the probability space, which is sufficient for our purposes. In general, a probability space is defined as a triple $(\Omega, \mathcal{F}, P)$, where Ω is the sample space, $\mathcal{F}$ is a σ -algebra of events (subsets of Ω), and P is a probability measure that assigns probabilities to events in $\mathcal{F}$.

An event A is any subset of the sample space:

$$\forall A \subseteq \Omega : \exists P(A) : \sum_{\omega \in A} P(\omega) = P(A)$$

8.1.2 Kolmogorov's Axioms of Probability

1. **All probabilities are between 0 and 1**

$$0 \leq P(A) \leq 1$$

2. **The probability of the entire sample space is 1, and the probability of the empty set is 0:**

$$P(\Omega) = 1, P(\emptyset) = 0$$

3. **The probability of a union of disjoint events is the sum of their individual probabilities:**

$$P(\bigcup_{i=1}^n A_i) = \sum_{i=1}^n P(A_i)$$

→ **The probability of any two events A and B is given by:**

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

→ **The probability of the complement of an event A is given by:**

$$P(\neg A) = 1 - P(A)$$

4. **Axioms restrict the set of beliefs that an agent can hold:**

e.g. A and $\neg A$ cannot both be true.

Just as in logic, the violation of the axioms leads to inconsistent beliefs that may result in nonsensical or wrong conclusions and actions.

8.1.3 Random Variables

Specific events are often a bit complicated to work with. A random variable is a function from outcomes in the sample space Ω to values in a codomain, e.g. $X : \Omega \rightarrow \mathcal{X}$.

Example: Roulette

- **Atomic Events ($\in \Omega$):** Numbers 0-36
- **Random Variables ($X \in \mathcal{X}$):**
 - Red | Black
 - Odd | Even
 - High | Low (1-18 | 19-36)
 - Street | Square | Split
 - ...

Then for every atomic event we can apply a random variable to check whether it is true (e.g. `red(36) = true`).

The probability of an event X taking a specific value x_i $P(X = x_i)$ is obtained by summing the probabilities of all atomic event ω that result in $X(\omega) = x_i$:

$$P(X = x_i) = \sum_{\omega: X(\omega)=x_i} P(\omega)$$

8.1.4 Propositions

A proposition is a disjunction of atomic events in which it is true:

$$(a \vee b) \equiv (\neg a \wedge b) \vee (a \wedge \neg b) \vee (a \wedge b) \\ \rightarrow P(a \vee b) = P(\neg a \wedge b) + P(a \wedge \neg b) + P(a \wedge b)$$

Syntax of Propositions

- **Propositional or Boolean** random can be true or false
 - `hasUmbrella = true` is a proposition and can be written as `hasUmbrella`
- **Discrete** random variables (finite or infinite)
 - e.g. Weather is one of {sunny, cloudy, rainy, snow}
 - `Weather = rainy` is a proposition
 - Values must be mutually exclusive and exhaustive (All possible cases are covered by values)
- **Continuous** random variables (bounded or unbounded)
 - e.g. Temperature
 - `Temp = 25.5` , `Temp > 23` are propositions

8.2 Joint Distribution

A joint distribution is the probability of combined events e.g. The probability that $X = x$ and $Y = y$ is true:

$$P(x, y) \equiv P(X = x \wedge Y = y)$$

Applying this to a whole set we can obtain a **joint probability distribution truth table**:

Smoking / Cancer	No	Benign	Malignant
No	0.768	0.024	0.008
Few	0.132	0.012	0.006
Many	0.035	0.010	0.005

Joint Probability Distribution of Smoking and Cancer

8.2.1 Marginalization

Often we don't want to talk about the joint distribution of two events, but get the marginal distribution of one event:

For any set of variables X and Y we can compute the probability

$$P(Y) = \sum_{i=1}^n P(x_i, Y)$$

Smoking	No	Benign	Malignant
No	0.768	0.024	0.008
Few	0.132	0.012	0.006
Many	0.035	0.010	0.005

Joint Probability Distribution of Smoking and Cancer

So for example:

$$P(Y = \text{few}) = P(\text{no}, \text{few}) + P(\text{benign}, \text{few}) + P(\text{malignant}, \text{few}) = 0.132 + 0.012 + 0.006 = 0.15$$

8.2.2 Conditional Probabilities

The probability of $X = x$ under the assumption that $Y = y$ is true:

$$P(x|y) = \frac{P(x \wedge y)}{P(y)} = \frac{P(x, y)}{P(y)}$$

The product rule yields an alternative representation for the joint probability:

$$P(x, y) = P(x|y) \cdot P(y) = P(y|x) \cdot P(x)$$

Expanding this onto the **Chain rule**:

$$\begin{aligned} P(X_1, \dots, X_n) &= P(X_1, \dots, X_{n-1})P(X_n|X_1, \dots, X_{n-1}) \\ &= P(X_1, \dots, X_{n-2})P(X_{n-1}|X_1, \dots, X_{n-2})P(X_n|X_1, \dots, X_{n-1}) \\ &= \prod_{i=1}^n P(X_i|X_1, \dots, X_{i-1}) \end{aligned}$$

8.2.3 Independence

X and Y are independent from another if one of the following is true:

$$\begin{aligned} 1 : & P(X|Y) = P(X) \\ 2 : & P(Y|X) = P(Y) \\ 3 : & P(X, Y) = P(X)P(Y) \end{aligned}$$

Independent variables are not affected by the other variable. This reduces the amount of possible variables.

However, absolute independent variables are rare.

8.2.4 Bayes Theorem

$$\underbrace{P(x|y)}_{\text{Posterior}} = \frac{\overbrace{P(y|x)}^{\text{Likelihood}} \overbrace{P(x)}^{\text{Prior}}}{\underbrace{P(y)}_{\text{Marginalization}}}$$

- **Posterior**: Probability of hypothesis X after evidence Y

- **Likelihood:** Probability of evidence Y given hypothesis X is true
- **Prior:** Probability of the hypothesis X before considering evidence Y
- **Marginalization:** Probability of evidence Y

8.3 Uncertainty in AI

Joint distribution is done by enumerating everything:

- **Worst-Case Run Time:** $O(2^n)$
 - n = number of random variables
- **Space Complexity:** $O(2^n)$ = size of the table

Due to this we mainly use **independencies** to compress the representation.

9 Bayesian Networks

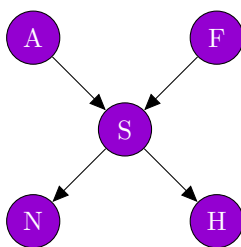
Bayesian Networks are a simple graphical notation for **conditional independence** assertions, hence for **compact specifications of full joint distributions**.

A Bayesian Network is **directed, acyclic graph** with

- **Nodes:** One node per variable
- **Edges:** A directed edge from node N_i to node N_j encodes a direct conditional dependence of N_j on N_i . It may represent a causal relationship, but it does not have to.

Set of random variables $\{X_1, \dots, X_n\}$

Directed Acyclic Graph (DAG)



Conditional Probability Distribution (CPD)

- Each random variable X_i in the network is associated with a CPD given its parents ($Pa(X_i)$)

$$P(X_i | Pa(X_i))$$

- Each node has a conditional probability distribution conditioned on its parents. If a node has no parents, then it is associated with a prior distribution.

Joint Distribution:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | Pa(X_i))$$

Local Markov Assumption:

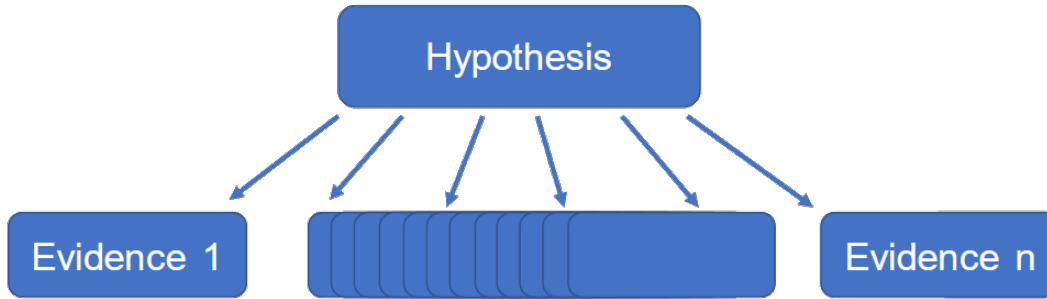
Each random variable X_i is conditionally independent of its non-descendants, given its parents.

$$X_i \perp \text{nonDescendants} | Pa(X_i)$$

9.1 Naïve Bayes

A **naïve Bayes** model assumes that all effects are independent given the cause:

$$P(H, E_1, \dots, E_n) = P(H) \cdot \prod_{i=1}^n P(E_i|H) \text{ } E \text{ is the evidence, } H \text{ is the hypothesis}$$



9.2 Inference in Bayesian Networks

Query $P(X|e)$

Definition of conditional probability: $P(X|e) = \frac{P(X, e)}{P(e)}$

Up to normalization: $P(X|e) \propto P(X, e)$

Can be rewritten as:

$$P(Y) = \underbrace{\sum_{X_i \notin Y}}_{\text{Marginalization}} \underbrace{\prod_{i=1}^n P(X_i | Pa(X_i))}_{\text{BN Semantics}}$$

9.2.1 Variable Elimination

Given a Bayesian Network and a query $P(X|e)/P(X, e)$.

Instantiate evidence e .

Choose an elimination order over the variables $X_1, \dots, X_n$.

Initial factors of probability distribution comprised of: $f_1, \dots, f_n$.

For $i = 1$ to n , if $X_i \notin \{X, E\}$:

Collect factors $f_1, \dots, f_k$ that contain X_i .

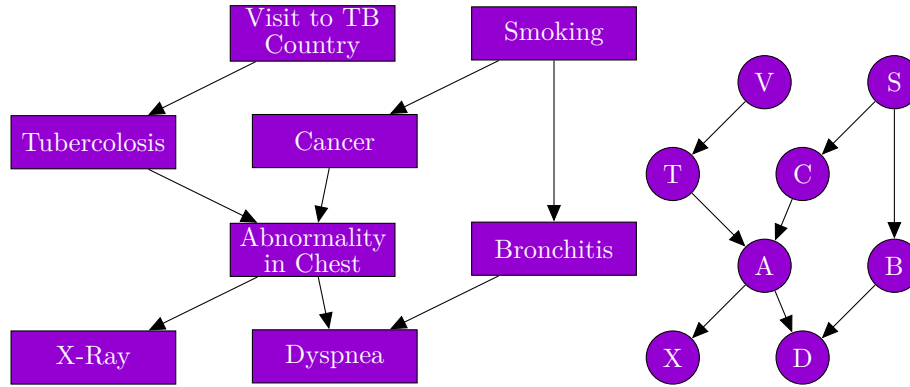
Generate a new factor by eliminating X_i from $f_1, \dots, f_k$:

$$g = \sum_{X_i} \prod_{j=1}^k f_j$$

Remove all factors $f_1, \dots, f_k$ and add new factor g to the network.

Normalize $P(X, e)$ to obtain $P(X|e)$.

Example



Assume we want to compute $P(d)$, so we need to **eliminate** v, s, t, c, a, b, x .

The **probability distribution** is given as the product of multiple factors:

$$P(v, s, t, c, a, b, x, d) = P(v)P(s)P(t|v)P(c|s)P(b|s)P(a|c, l)P(x|a)P(d|a, b)$$

Lets choose the elimination order: v, s, x, t, c, a, b

From that we get:

	$P(v, s, x, t, c, a, b, d)$	$= P(v)P(s)P(t v)P(c s)P(b s)P(a c, l)P(x a)P(d a, b)$
$v \Rightarrow$	$P(s, x, t, c, a, b, d)$	$= f_v(t)P(s)P(c s)P(b s)P(a c, l)P(x a)P(d a, b)$
$s \Rightarrow$	$P(x, t, c, a, b, d)$	$= f_v(t)f_s(b, c)P(a t, c)P(x a)P(d a, b)$
$x \Rightarrow$	$P(t, c, a, b, d)$	$= f_v(t)f_s(b, c)f_x(a)P(a t, c)P(d a, b)$
$t \Rightarrow$	$P(c, a, b, d)$	$= f_s(b, c)f_x(a)f_t(a, c)P(d a, b)$
$c \Rightarrow$	$P(a, b, d)$	$= f_x(a)f_c(a, b)P(d a, b)$
$a \Rightarrow$	$P(b, d)$	$= f_a(b, d)$
$b \Rightarrow$	$P(d)$	$= f_b(d)$

This unfortunately is not efficient.

Theorem

In general, exact inference in Bayesian Networks is computationally intractable (NP-hard).

9.2.2 Approximate Inference by Stochastic Sampling

Basic Idea:

1. Draw N samples from a sampling distribution S
2. Compute an approximate posterior probability $\hat{P}$
3. Show this converges to the true probability P

Draw samples

Given:

- Random Variable $X|D(X) = \{0, 1\}$
- $P(X) = \{0.3, 0.7\}$ ($P(X=0) = 0.3$, $P(X=1) = 0.7$)

Sample $X = P(X)$

- Get a random number $r \in [0, 1]$
- If $r < 0.3$ then $X = 0$
- Else $X = 1$

Can be generalized to any domain size.

Sampling from "Empty Network"

Ergo, **generating samples from a network that has no evidence associated with it.**

Basic Idea:

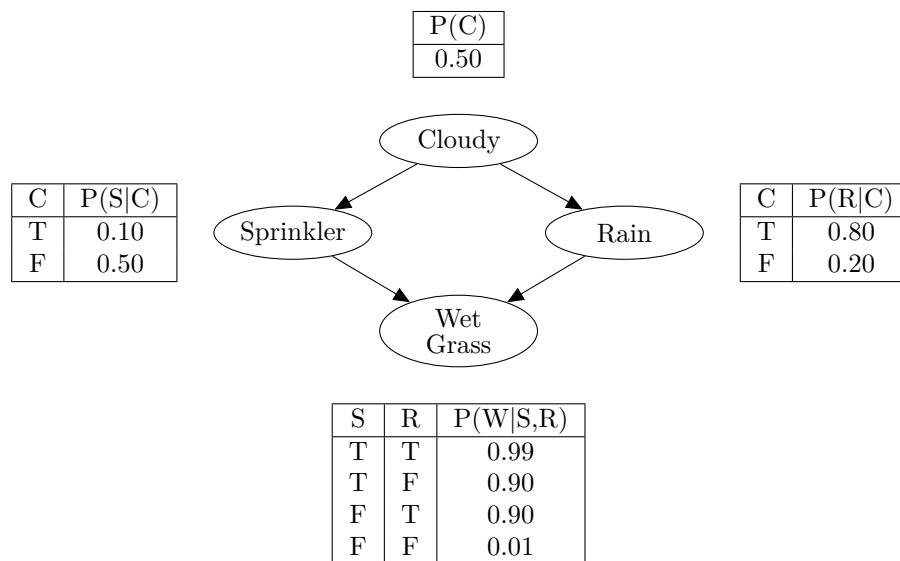
- Sample a value for each variable in topological (in respect to dependencies) order
- Using the specified conditional probabilities

```

1 // belief network specifies joint distribution  $P(X_1, \dots, X_n)$ 
2 Function prior_sample(belief_network)  $\rightarrow$  event sampled from belief network:
3   x = event with n elements;
4   For  $i = 1$  to  $n$  do
5      $x_i$  = random sample from  $P(X_i | Pa(X_i))$  given the values of  $Pa(X_i)$  in x;
6   return x

```

Example:



Bayesian Network for Weather and Wet Grass

Probability Estimation using Sampling

Calculating a probability estimation:

- Sample many points using the algorithm above
- Count how often each possible combination $x_1, \dots, x_n$ occurs

- Estimate the probability by the observed percentages

$$\hat{P}(x_1, \dots, x_n) = N_{PS}(x_1, \dots, x_n) / \text{number of samples}$$

This converges towards the joint probability function.

Markov Chain Monte Carlo (MCMC) Sampling

```

1 Function mcmc_ask( $X, e, \text{belief\_network}, \text{num\_samples}$ )  $\rightarrow$  estimate of  $P(X|e)$ :
2   countX = [] // number of times each X occurs, initially 0 for all
3   Z = [non-evidence variables] // list of non-evidence variables
4   x = e // current state of the network, initially e
5   initialize non-evidence values in x with random values;
6   // Gibbs sampling
7   For j=1 to num_samples do
8     ForEach  $Z_i \in Z$  do
9       [ x[Zi] = sample from  $P(Z_i | \text{markov\_blanket}(Z_i))$ 
10    countX[X = x[X]] += 1 // x is the value of X in x
11  return normalize(countX)

```

More samples result in better approximates.

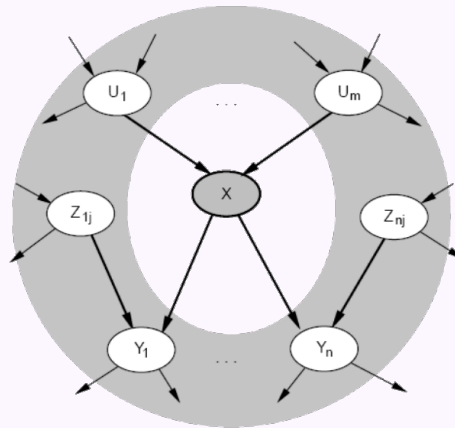
Markov Blanket

A Markov Blanket is the minimal set of variables $MB(X)$ such that X is conditionally independent of all other variables in the network given $MB(X)$. It consists of **parents (direct causes)**, **children (direct effects)** and **childrens parents (co-causes)**. Alternatively: A markov blanket includes all variables that **directly influence** or **are influenced** by a variable X . Everything outside of the markov blanket is irrelevant to X , if we know all direct influences. This makes it easier to compute probabilities.

$$P(X|MB(X), O) = P(X|MB(X))$$

$$MB(X) = \{U_1, \dots, U_m, Y_1, \dots, Y_n, Z_{1j}, \dots, Z_{nj}\}$$

O is all variables outside of $MB(X) \cup X$



Gibbs Sampling

Basic Idea:

1. **Initialize** all variables with random values
2. **Iterate through each variable**, updating it based on Markov Blanket
3. **Repeat** until samples converge asymptotically to the target posterior

Gibbs Sampling utilized Markov Blankets by reducing the number of variables that need to be considered at each step.

Example:

Estimate $P(\text{Rain}|\text{Sprinkler} = \text{true}, \text{WetGrass} = \text{True})$

1. Sample Cloudy or Rain given its Markov Blanket, repeat n times
2. Count number of times Rain is true and false in the samples

E.g. sample 100 states and count 31 times Rain and 69 times not Rain.

$$P(\text{Rain}|\text{Sprinkler} = \text{true}, \text{WetGrass} = \text{True}) = \text{Normalize} \langle 31, 69 \rangle = \langle 0.31, 0.69 \rangle$$

Theorem

Chain approaches stationary distribution:

The long-run fraction of time spent in each state is exactly proportional to the posterior probability.

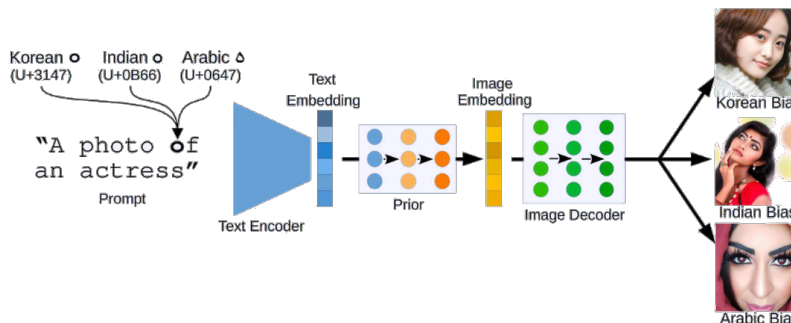
If you're interested in more details about Bayesian Networks, Markov Methods and other probabilistic graphical models, and their applications, I highly recommend the course **Probabilistic Graphical Models**.

10 Machine Ethics

10.1 Concerns

10.1.1 Biases

AI systems are inherently biased, because the data they're trained on is too. One small detail can yield entirely different results.



Bias in Text-to-Image

10.1.2 Data Privacy

Due to the large amounts of data an AI is usually trained on, it is almost inevitable that this data, and by extension the AI model, contains information that it shouldn't have. This may include leaked personal information and the likes, but also things like simply being able to recognize people it isn't trained for.

If we wanted to train a model to be able to recognize one specific person we also have to feed it data that doesn't contain that person. This causes the program to also being able to detect other persons.

10.1.3 Causal Reasoning without Causality

LLMs work on a mostly probabilistic basis. This means that they calculate the string of words that is most likely to correspond to the input. By doing this they mimic the causal reasoning of the data they're trained on, but do not actually employ causal connections.

This leads to a false sense of security when asking LLMs about factual information.

10.2 Mitigation of Concerns

10.2.1 Make AI systems aware of unwanted behaviour

We can try to mitigate the problem of biases by trying to train AI on data that is as unbiased as possible. This, of course, is incredibly hard to do, as AI models need a large amount of data, which is hard to moderate.

Another option is to implement filters. For example, we can implement a filter that makes it impossible to produce pornographic pictures of specific persons, or try to "rework" the result into something better.

Another thing we can do is **train models to be able to detect biases and concerns** (revision engine). Once again, this is hard to do, as first you need to find out what biases and concerns exist, and then train models to detect them and answer responsibly. This results in a long process, that is often still ongoing when an AI is deployed and continues until it is discontinued.

Often **reasoning** models are used for this. If we are able to make an AI be able to explain the reasoning behind its answer, it is more likely to detect when it is doing something it shouldn't.

11 Machine Learning & Neural Networks

11.1 Learning

Learning is an essential process for dealing with unknown environments. An agent is almost **never omniscient**.

Learning also makes an agent "aware" of their environment. Rather than trying to change the environment so that they can take their actions, they might **try to use and adapt to the environment** to fulfill their tasks.

Lastly, Learning improves the agents decision mechanisms the longer it is active. No longer will an agent repeat actions hoping for different results (e.g. local extrema).

11.1.1 Methods of Learning

Memorization:

Accumulates individual facts and their outcomes. This is limited by the time to observe facts and memory to store facts.

Better: **Generalization:**

Deduce new facts from old facts. Limited by accuracy of deduction process, assumes relations between past and future.

11.1.2 Inductive Learning

Inductive learning is the simplest form of learning. It deduces a trend from examples.

Basic Idea:

- f is the (unknown) target function. $f(x)$ is then called the target
- Examples are defined in the form $(x, f(x))$ e.g. Weather data: (1, Rain)
- Use examples to find a hypothesis h such that $h \approx f$

Hypothesis h is **consistent** if $\forall (x_i, y_i) \in D : h(x_i) = y_i \quad \text{i.e. } h(x_i) = f(x_i)$.

Ockhams Razor

The **best explanation** is the **simplest explanation** (i.e. fewest assumptions) that fits the data

Overfitting Avoidance

Maximize the combination of consistency and simplicity.

11.2 Machine Learning

Programming an algorithm to automatically learn a program from data or experience.

Machine Learning is **not synonymous** with AI. Many AI algorithms are based on Machine Learning, but not all.

11.2.1 Machine Learning & Human Learning

Human learning is often

- very data- and knowledge efficient
- a complete multitasking, multi-modal system
- time-inefficient

Machine learning is often inspired by human learning, the goal however, is not to recreate human learning.

- May borrow ideas from human learning (e.g. neural networks)

- May perform better or worse, depending on the task, available data and model architecture

11.2.2 Designing a Learning System

Machine Learning is not the solution to every problem!

1. Do I need a learning approach for my problem?
 - Is there a pattern to detect?
 - Can I solve the problem analytically?
 - Do I have data to train on?
2. What type of problem do we have?
 - How to represent it?
 - Which algorithm to use?
3. Gather and organize data
 - Preprocessing is important
4. Fitting / Training your model
5. Optimization
6. Evaluate and iterate back to step 2

11.2.3 Types of Learning

Supervised Learning

Learning based on labeled datasets. Learns to map inputs to outputs based on pairs in the dataset.

Unsupervised Learning

Learning based on unlabeled datasets. Searches for patterns and similarities in the data.

Reinforcement Learning

Agent acts and receives positive or negative reward. The agent learns to act in a way that maximizes the reward.

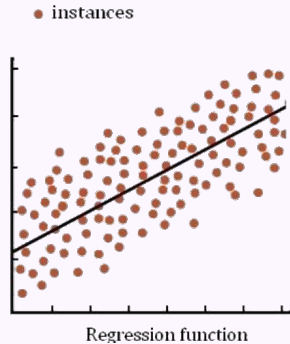
11.2.4 Supervised Learning

Given: Dataset $(x_1, y_1), \dots, (x_n, y_n)$

Goal: Find a function h such that $h(x_i) \approx y_i$

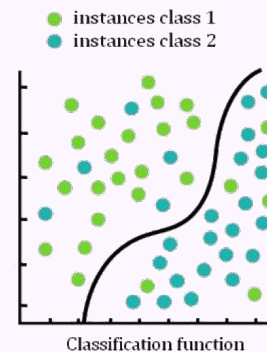
Regression Task

- y is a continuous value
- Example: Temperature tomorrow



Classification Task

- y is a discrete class label
- Model predicts a discrete class label, often via class scores or estimated class probabilities
- Example: Will it be above 0°C tomorrow?



11.2.5 Representation

Machine learning algorithms need to handle a lot of different data (e.g. images, audio, etc.).

To make this as easy as possible, one option is to represent our input as an **input vector** (in $\mathbb{R}^n$). Vectors are a great representation as we can transform the data using linear algebra. Other representations include tensors, graphs, sets, structured symbolic data, etc.

Representation

Mapping input to another space, that is **easier** to manipulate.

Feature

Features are the **independent variables** in machine learning models.

Model

The representation of what our algorithm has learned from the data it used in training. The model is the **output representation** of the learned "rule set".

11.2.6 Feature Engineering

Feature engineering is the process of selecting, manipulating and transforming raw data into features that can be used for machine learning.

For this it is important to know your data:

- Data distribution
- Outliers
- Data reflective of reality

- Biased data
- ...

On the basis of these factors we can choose approaches that best fit our needs.

Regression:

- Linear Regression
- Multiple Linear Regression
- Regression Trees
- Non-linear Regression
- Polynomial Regression
- ...

Classification:

- Random Forest
- Decision Trees
- Logistic Regression
- Naïve Bayes
- Support Vector Machines
- ...

11.3 Evaluating a Model

To evaluate a model we need to know the goal we are trying to achieve.

Based on the goal we can use different evaluation metrics. These are the most common ones:

- Accuracy
- Precision
- Recall
- Mean Squared Error
- ...

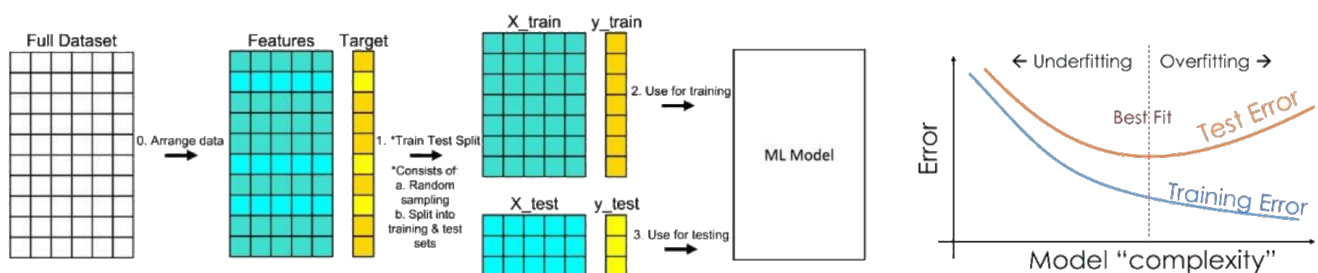
Which metric is best depends on the problem we are trying to solve.

11.3.1 Overfitting

Overfitting describes the problem, that a model is trained "too well" so that its closer to memorization than generalization. This means that the model is not able to generalize to new data and therefore performs worse on new data.

To find out if a model is overfitting we can use the following steps:

- Splitting data
 - Split data into training and test data
- Train the model on the training data
- Evaluate the model on the test data
- If the model performs well on the training data but poorly on the test data, we have overfitting



To avoid overfitting we can use the following methods:

- Regularization
- Use more data, augmented data (adding noise)
- Select different features

- Cross validation
- Ensemble methods

These will not be discussed in detail here, check out other advanced machine learning courses for more information. (DMML, DLAM, etc.)

11.4 Neural Networks

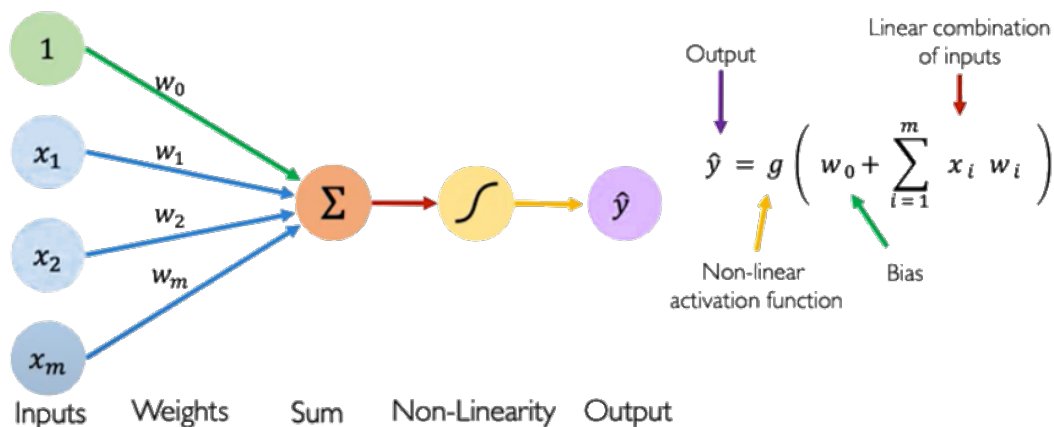
11.4.1 Deep Learning

Up until now we always had to supervise learning in some way, like feature extraction.

But with deep learning we can **learn the underlying features** directly from data without specifying them.

11.4.2 Perceptron

The perceptron can be seen as an **artificial neuron**. It takes multiple inputs, that are weighed individually and summed up to give a single output.



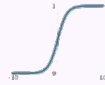
- Neurons correspond to nodes or units
- A link from unit j to unit i propagates activation y from j to i
- The weight $w_{i,j}$ of the link determines the strength and sign of the connection
- All weights together are called W or θ and describe the model
- The **total input activation** is the sum of the input activations
- The **output activation** is determined by the activation function g

Activation Function

A activation function decides if a neuron should be active. Nowadays they're mostly non-linear.

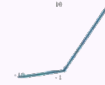
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



Leaky ReLU

$$\max(0.1x, x)$$



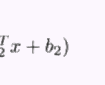
tanh

$$\tanh(x)$$



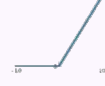
Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$



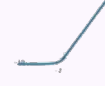
ReLU

$$\max(0, x)$$



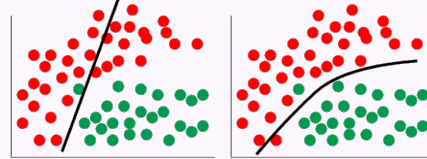
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Why do we need an activation function?

- It adds non-linearity to a neural network
- Without it we would have a linear regression model
- Differentiable activation functions are needed for backpropagation, see later in this section.



Activation Function Derivatives

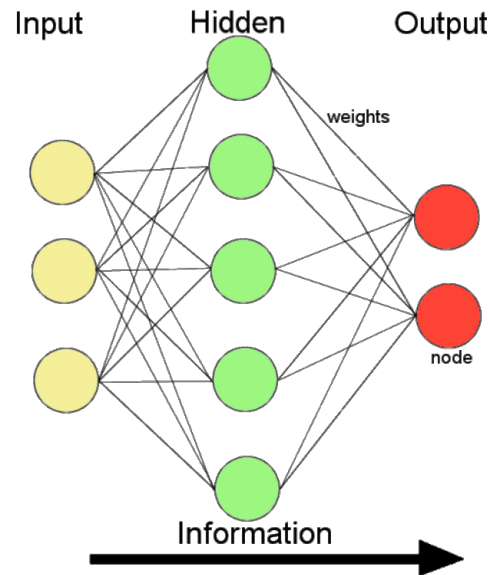
The derivative of an activation function is needed for backpropagation, see later in this section. Common ones:

Name	Function	Derivative
Sigmoid	$g(x) = \frac{1}{1+e^{-x}}$	$g'(x) = g(x)(1 - g(x))$
Tanh	$g(x) = \tanh(x)$	$g'(x) = 1 - g^2(x)$
ReLU	$g(x) = \max(0, x)$	$g'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$
Leaky ReLU	$g(x) = \begin{cases} x & x > 0 \\ 0.01x & x \leq 0 \end{cases}$	$g'(x) = \begin{cases} 1 & x > 0 \\ 0.01 & x \leq 0 \end{cases}$
Maxout	$g(x) = \max(x_1, x_2, \dots, x_n)$	$g'(x) = \begin{cases} 1 & x_1 = \max(x_1, x_2, \dots, x_n) \\ 0 & \text{otherwise} \end{cases}$
ELU	$g(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$	$g'(x) = \begin{cases} 1 & x > 0 \\ \alpha e^x & x \leq 0 \end{cases}$

11.4.3 Perceptron to Neural Network

- Perceptrons may have multiple output nodes which can be combined to other perceptrons
- We can build networks of these nodes: **Multilayer Perceptron (MLP)**
- In a MLP information flow is **unidirectional**
- Information is distributed and processed in parallel
- For each node:

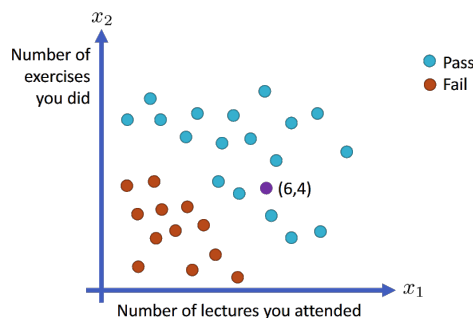
$$z_{k,i} = \sigma(w_{0,i}^k + \sum_{j=1}^{n_{k-1}} z_{k-1,j} \cdot w_{j,i}^k)$$



11.5 Forward Propagation

11.5.1 Applying a NN

Assume a distribution of an exam:

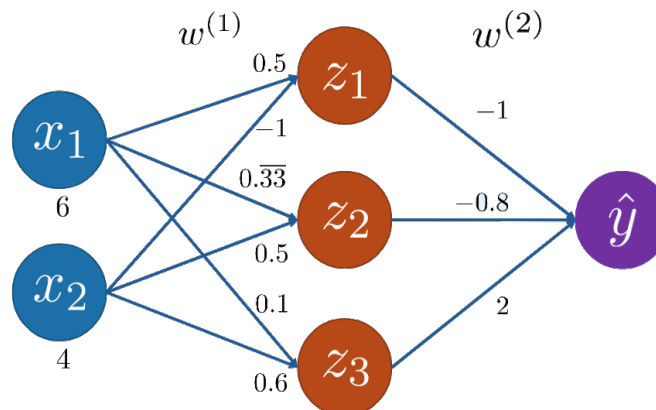


Assume the purple point is $x_1 = 6, x_2 = 4$ and the weights of our NN are:

$$\begin{array}{lll} w_{1,1}^1 = 0.5 & w_{2,1}^1 = -1 & w_{1,1}^2 = -1 \\ w_{1,2}^1 = 0.33 & w_{2,2}^1 = 0.5 & w_{2,1}^2 = -0.8 \\ w_{1,3}^1 = 0.1 & w_{2,3}^1 = 0.6 & w_{3,1}^2 = 2 \end{array}$$

Further assume a bias $w_{0,i}^k = 0$.

This yields a NN model:



From this we can calculate the outputs of each neuron by applying the neurons activation function to the weighted sum of the inputs, and propagating the result to the next layer. This is called **forward propagation**.

For the first layer (z) we have:

$$\begin{aligned} z_1 &= g\left(w_{0,1}^1 + \sum_{j=1}^2 x_j \cdot w_{j,1}^1\right) = g\left(x_1 w_{1,1}^1 + x_2 w_{2,1}^1\right) = g(3 + (-4)) = g(-1) \\ z_2 &= g(2 + 2) = g(4) \\ z_3 &= g(0.6 + 2.4) = g(3) \end{aligned}$$

For the activation function g we use the sigmoid function: $g(x) = \frac{1}{1+e^{-x}}$

This yields $z_1 = 0.26, z_2 = 0.98, z_3 = 0.95$.

To get the prediction $\hat{y}$ we need to calculate the output activation:

$$\begin{aligned} \hat{y} &= \sigma\left(w_{0,1}^2 + \sum_{j=1}^3 z_j \cdot w_{j,1}^2\right) \\ &= \sigma\left(w_{0,1}^2 + z_1 w_{1,1}^2 + z_2 w_{2,1}^2 + z_3 w_{3,1}^2\right) \\ &= \sigma((-1) \cdot 0.26 + (-0.8) \cdot 0.98 + 2 \cdot 0.95) \\ &= \sigma(0.856) = 0.7 \end{aligned}$$

With this we can conclude, that according to our model, the student has a 70% chance of passing the exam.

11.6 Backpropagation

Assume different weight than in the example above so that $\hat{y} = 0.14$.

Quantifying Loss

$$\mathcal{L}(f(x^i; \theta), y^i)$$

Describes the cost of incorrect predictions.

Empirical Loss

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^i; \theta), y^i)$$

Measures the total loss over the dataset. Also known as **objective function, cost function or empirical risk**.

Our goal is to **minimize the empirical loss**:

$$\theta^* = \arg \min_{\theta} J(\theta)$$

θ^* is the weight setting where the empirical loss is minimized.

This can be done using **Gradient descent**, as the **Weight space** is a N-dimensional space where N is the total number of weights.

Algorithm:

1. Initialize weights randomly
2. Loop until convergence
 - Compute gradient $\nabla J(\theta) = \frac{\partial J(\theta)}{\partial \theta}$
 - Update weights $\theta \leftarrow \theta - \alpha \nabla J(\theta)$ (with α learning rate)
3. Return weights

Backpropagation: Computing $\nabla J(\theta) = \frac{\partial J(\theta)}{\partial \theta}$

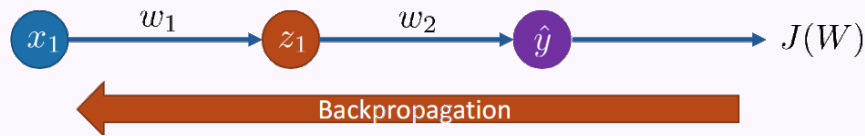
Backpropagation is the algorithm used to compute the gradients for gradient descent. It works by applying the chain rule of calculus to compute the gradient of the loss function with respect to each weight in the network, starting from the output layer and moving backwards through the hidden layers to the input layer:

1. **Forward pass:** Compute the output of the network for a given input and calculate the loss.
2. **Backward pass:** Compute the gradient of the loss with respect to each weight by propagating the error backwards through the network:
 - For the output layer, compute the gradient of the loss with respect to the output of the network and then with respect to the weights connecting the last hidden layer to the output layer:

$$\frac{\partial J}{\partial w_L} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_L}$$

- For all other layers, compute the gradient of the loss with respect to the output of the layer and then with respect to the weights connecting the previous layer to the current layer:

$$\frac{\partial J}{\partial w_k} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_k} \frac{\partial z_k}{\partial w_k} = \frac{\partial J}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_k} = \text{Error at current neuron} \cdot \text{Error of previous layer}$$



Example



Given $g = \text{ReLU}(x) = \max(0, x)$, $g'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$, $x_1 = 4$, $y = 1$, $\hat{y} = 2$, $\mathcal{L} = (\hat{y} - y)^2$, $J(\theta) = \mathcal{L}$

ReLU: Rectified Linear Unit

If x is greater than 0 return x else return 0.

Goal: $\frac{\partial J(\theta)}{\partial w_1} = \frac{\partial J(\theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1}$

1. $\frac{\partial J(\theta)}{\partial \hat{y}} = 2(\hat{y} - y) = 2(2 - 1) = 2$ (Power rule)
2. $\frac{\partial \hat{y}}{\partial w_1} = \text{ReLU}'(w_0 + \sum_{i=1}^n w_i x_i) \cdot x_1 = 1 \cdot 4 = 4$
3. $\frac{\partial J(\theta)}{\partial w_1} = \frac{\partial J(\theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1} = 2 \cdot 4 = 8$

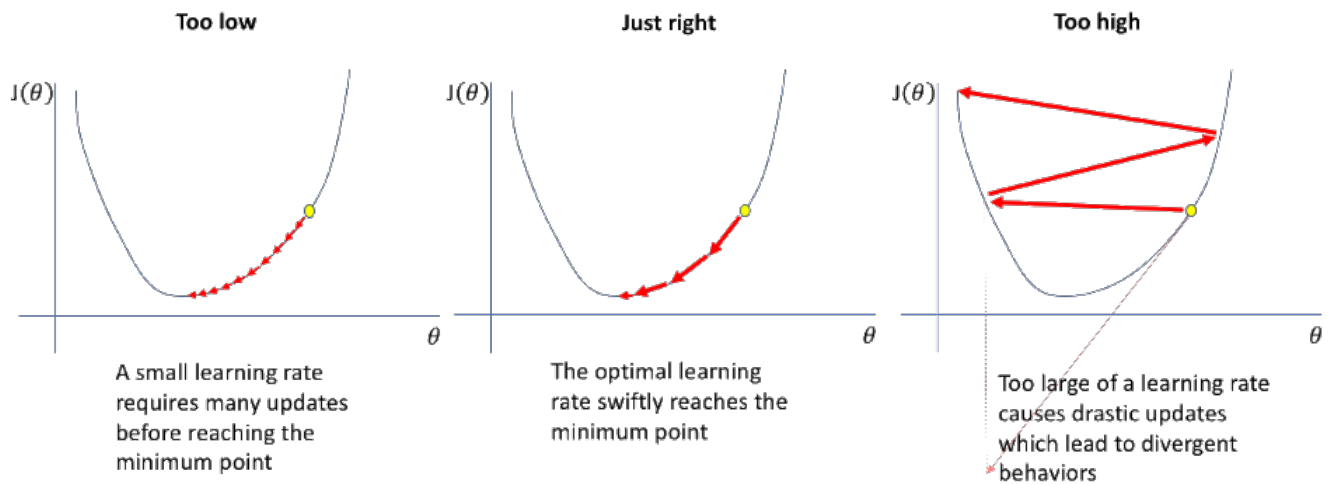
Now we also need to update the weights $\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$ (assume learning rate $\alpha = 0.05$):

$$\begin{aligned} \theta_{\text{new}} &= \theta_{\text{current}} - \alpha \frac{\partial J(\theta)}{\partial \theta} \\ \theta_{\text{new}} &= 0.5 - 0.05 \cdot 8 = 0.1 \end{aligned}$$

Loss function in general are hard to optimize as its difficult to find a global minimum.

Basic Idea:

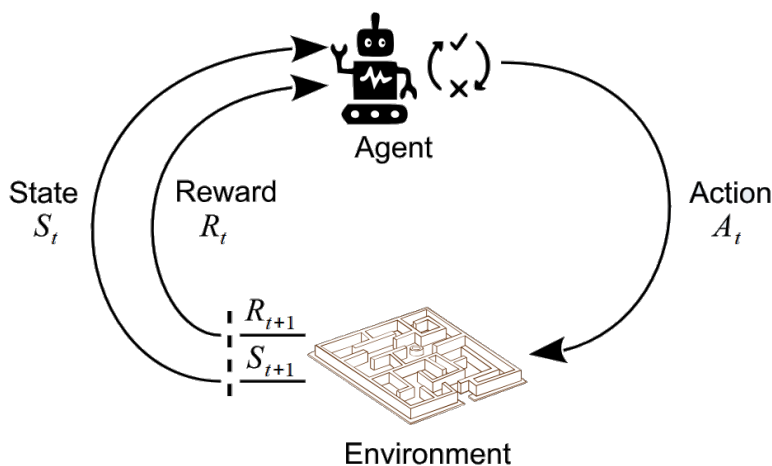
Change weight into the direction of the steepest descent of the error function given a step size - This step size is called the **learning rate α** , but finding a good value is hard.



12 Reinforcement Learning

In Reinforcement Learning we assign each action a **positive or negative reward** depending on what we want the agent to do.

Reinforcement Learning algorithms attempt to find a policy (what action to do in which state) for **maximizing the cumulative reward** for the agent.



Interaction loop between agent and environment

Mathematically this is typically represented by a **Markov Decision Process (MDP)**.

Reinforcement learning differs from supervised learning, that we never explicitly show a desired outcome, we merely give feedback. Additionally we never correct sub-optimal actions.

One of the biggest problems in RL is how we assign rewards to actions. A lot of the time we do not assign a reward to just one action as that would be very inefficient, but rather to a sequence of actions.

- Given a sequence of states and actions how do we define which actions were effective and ineffective?
- Actions might not immediately lead to a reward, they might have an effect that's only apparent later on
- Not every environment is deterministic, the same action might have different results
- Agents are not perfect - an action might not always be performed correctly

12.1 Markov Decision Process (MDP)

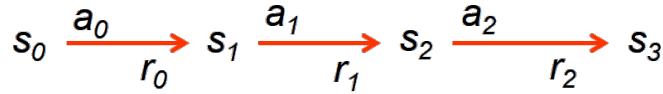
A simple MDP is defined by a 5-tuple (S, A, T_a, R_a, γ) , where:

- **S** : A set of states $s \in S$
- **A** : A set of actions $a \in A$
- **T_a** : Transition function $T(s, a, s')$
 - Probability that a from s leads to s'
 - i.e. $P(s' | s, a)$ also called the model
- **R_a** : Reward function $R(s, a, s')$
 - Reward for taking action a in state s and ending up in state s'

– Sometimes just $R(s)$ or $R(s')$

- **Discount factor γ** : How much we discount future rewards

Additionally, it has an initial state distribution $P(s_0)$ and a terminal state s_T (Termination condition).



Markov Property

The probability of moving to the next state depends only on the current state and action, **not on past states and actions**.

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

The future is independent of the past, given the present

12.1.1 Solve MDPs

To solve an MDP we want to compute an **optimal policy** $\pi^* : S \rightarrow A$.

- A policy maps states to distributions over (or directly) actions.
- An optimal policy maximizes expected utility
- Defines a reflex agent

Deterministic Policy

A policy that deterministically maps each state to a **single action**.

$$a_t = \pi(s_t)$$

Stochastic Policy

A policy that maps each state to a **distribution** over actions.

$$a_t \sim \pi(\cdot | s_t)$$

At any time t , the agent tries to select an action to maximize the sum G_t (**Return**) of discounted rewards received in the future:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

We discount **future rewards** with the discount factor γ to improve multiple things:

- It reflects the uncertainty of the future
- It makes the problem mathematically tractable / Finite mathematical returns
- It encourages the agent to find policies that lead to earlier rewards

$\gamma \in [0, 1]$ is often set to 0.9.

For each state we calculate a **utility value** equal to the sum of future rewards.

12.1.2 Optimal Utility

Optimal utilities define optimal policies:

- **Value function**:

- Defines the value of a state s :
- $V^*(s)$ = expected utility starting in s and thereafter acting optimally
- **Action-Value Function:**
 - Defines the value of a q-state (s,a) :
 - $A^*(s,a)$ = expected utility starting in s , taking action a and thereafter acting optimally
- Define **optimal policy π^*** :
 - $\pi^*(s)$ = optimal action from state s

A policy is **better than or equal** to another policy if its expected return is greater than or equal to the other policy **for all states**.

There is always at least one optimal policy π^* that is better than or equal to all other policies.

All optimal policies share the **same optimal state-value function v^*** , which gives the maximum expected return for any state s over all possible policies.

All optimal policies share the **same optimal action-value function q^*** , which gives the maximum expected return for any state-action pair (s,a) over all possible policies.

Formalization

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s'|s, \pi(s)) V^\pi(s')$$

- V^π : Value of state s when acting according to π
- $R(s, \pi(s))$: Immediate reward
- γ : Discount factor
- $T(s'|s, \pi(s))$: Transition probability to arrive in s'

12.2 Reinforcement Learning

Reinforcement learning assumes an MDP:

- **Set of states $s \in S$**
- **Set of actions $a \in A$**
- **Model $T(s, a, s')$**
- **Reward function $R(s, a, s')$**
- **Discount factor γ**

The twist: In passive reinforcement learning, the agent follows a fixed policy and estimates its value. In active reinforcement learning, the agent must explore the environment to find the optimal policy / learn.

Model-based Approach

Learn (or use) the model and use it to derive the optimal policy

Model-free Approach

Derive the optimal policy without learning the model

On-Policy

An on-policy agent only learns about the policy it is executing

Off-Policy

An off-policy agent learns about a policy different from the one it is executing. Can learn the optimal policy regardless of the policy it follows during learning

Passive Learning

The agent simply watches the world and tries to learn the utilities of the states and actions. The goal is to execute a fixed policy (sequence of actions) and evaluate it

Active Learning

The agent also acts. The goal is to act and learn an optimal policy.

Sample-Based Policy Evaluation

Improve estimate of V by evaluating an average over multiple samples rather than a single action:

Return Average

$$V^\pi(s) \leftarrow \frac{1}{n} \sum_{i=1}^n G_i$$

Incremental Update

$$V^\pi(s_{t+1}) \leftarrow V^\pi(s_t) + \alpha(G_t - V^\pi(s_t))$$

Temporal-Difference (TD) Learning

Instead of waiting for full episode completion, we update the value function $V(s)$ each time s occurs.

$$V^\pi(s_{t+1}) \leftarrow V^\pi(s_t) + \alpha(G_t - V^\pi(s_t))$$

α is the learning rate.

Q-Learning

Q-Learning keeps track of samples and their Q-values, so that in the future it knows better which action to take in which situation.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[R_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

Q-Learning yields great results if enough exploration is done and if the learning rate is small enough. For the exploration, typically every relevant state-action pair must be visited infinitely often in order to guarantee convergence to the optimal policy. In practice, that is not feasible, but Q-Learning still works well in many situations.

However, Q-Learning is very space-inefficient as it keeps a table of all q-values. Additionally in most realistic situations it is impossible to learn about every single state - Too many states to visit and too many states to hold in memory.

12.2.1 Exploration

How do we choose which action to take next?

- Simplest: random actions (ϵ greedy)
 - Every time step, "flip a coin"

- With probability ε , act randomly
- With probability $1 - \varepsilon$, act according to current policy
- Problem with random action:
 - Random movement might not yield ideal results
 - One solution: lower ε over time
 - Another solution: exploration functions

12.2.2 Deep Q-Networks (DQN)

Similar to Q-Learning, but instead of using a Q-table to keep track of actions we train a deep neural network to estimate the best action. The DQN approximates $Q(s, a)$, the expected return of each action.

Uses ε -Greedy policy to select actions, essentially selecting $\arg \max_a Q(s, a)$ (with random exploration).

The learning process is similar to training a neural network using **backpropagation**.

12.2.3 Policy Search

Often feature-based policies that work well aren't the ones that approximate V / Q best.

To remedy this, we can try to learn the policy that maximizes the rewards rather than the value that predicts rewards.

Simple Policy Search:

- Start with an initial value function or q-function
- Nudge each feature weight up and down and see if the policy improves

Problems:

- How do we tell if a policy got better?
- Many sample episodes needed
- A lot of features make this harder

12.3 Deep Neural Networks (AlphaZero)

AlphaZero combines Neural Networks with Tree Search.

Example, Monte Carlo Tree Search (MCTS):

MCTS is used to guide move selection by searching through possible future moves before making a decision:

1. Selection: Starting from root (current state), moves are selected based on prior visit counts and a balance between exploration and exploitation
2. Expansion: When an unexplored move is encountered, a new node is added to the tree
3. Evaluation: Neural network predicts the policy (move probabilities) and value (expected outcome) for the new node
4. Backpropagation: The value is propagated back up the tree to update the estimated win probabilities.

After multiple simulation, AlphaZero picks the move with the highest visit count, rather than the highest policy probability.

Hereby the search depth and breadth might be limited using the policy prediction.

12.3.1 Training Objective: Loss Formulization

$$l(\theta) = \underbrace{\alpha(z - v)^2}_{\text{Mean Squared Error}} - \underbrace{\pi^T \log p_\theta(s)}_{\text{Cross Entropy}} + \underbrace{c||\theta||_2^2}_{\text{Regularizer}}$$

- α : value loss factor
- z : target value
- v : predicted value
- π^T : MCTS simulation distribution
- p : policy head output
- c : L_2 regularization constant